



2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : COMPUTER SCIENCE

**Design and Implementation of Qatra: A Microservices-
Based Platform for Blood Donation and Emergency
Response Management**

By: *Mourad Slim Jnayeh*

Academic supervisor: *Amel Ben Ncira*

Corporate Internship Supervisor: *Fahmi Mrabti*

keyrus

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Mourad Slim Jnayah

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Fahmi Mrabti

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname :Amel Ben Ncira

Signature / Signature

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to everyone who contributed to the completion of this end-of-studies project.

I am particularly grateful to my company supervisor, Mr. Fahmi Mrabti, for his availability, technical guidance, and constructive feedback throughout the internship. His support helped me approach architectural choices, delivery constraints, and engineering decisions with greater rigor and autonomy.

I also extend my sincere thanks to my academic supervisor, Ms. Amel Ben Ncira, for her methodological guidance and careful follow-up. Her advice contributed to structuring this work as a coherent academic and engineering project.

I thank Keyrus MEA for providing a professional and stimulating environment in which I could strengthen my software architecture, frontend, backend, DevOps, and project-management skills. I am grateful to the colleagues who shared their experience and encouraged continuous improvement.

Finally, I thank the faculty and staff of ESPRIT, as well as my family and friends, for their encouragement, patience, and unwavering support.

This report presents the design and implementation of Qatra, a distributed platform intended to improve blood-donation coordination and emergency response. The solution connects donors, donation-center staff, center administrators, and platform administrators through unified workflows for registration, health profiling, appointment scheduling, eligibility follow-up, emergency creation, donor matching, and real-time notification. The platform follows a microservices-oriented architecture centered on two Java 21 and Spring Boot services, an Angular 20 web application, PostgreSQL persistence, Kafka event exchange, Redis, and a containerized operational environment. Kubernetes manifests, Nginx routing, GitHub Actions, Prometheus, Grafana, Loki, Promtail, and Jaeger support deployment and observability concerns. Kanban was used to organize incremental delivery and maintain traceability between requirements, implementation, and validation. The report emphasizes the domain architecture, emergency matching logic, event-driven notification pipeline, security model, and the limits of the available verification evidence. Qatra demonstrates how modern distributed-system practices can support a socially significant domain while preserving maintainability, auditability, and operational visibility.

Keywords: blood donation, emergency response, microservices, Spring Boot, Angular, Kafka, Kubernetes, Kanban, observability.

Ce rapport présente la conception et la réalisation de Qatra, une plateforme distribuée destinée à améliorer la coordination des dons de sang et la réponse aux urgences. La solution relie les donneurs, le personnel des centres, les administrateurs de centres et les administrateurs de la plateforme à travers des parcours unifiés d'inscription, de profil médical, de prise de rendez-vous, de suivi d'éligibilité, de création d'urgence, de mise en correspondance des donneurs et de notification en temps réel. La plateforme adopte une architecture orientée microservices comprenant deux services Java 21 et Spring Boot, une application web Angular 20, PostgreSQL, Kafka, Redis et un environnement opérationnel conteneurisé. Des manifestes Kubernetes, Nginx, GitHub Actions, Prometheus, Grafana, Loki, Promtail et Jaeger couvrent le déploiement et l'observabilité. La méthode Kanban structure la livraison incrémentale et la traçabilité entre exigences, réalisation et validation. Le rapport met l'accent sur l'architecture métier, le moteur de mise en correspondance d'urgence, les notifications événementielles, la sécurité et les limites des preuves de validation disponibles.

Mots-clés : don de sang, réponse d'urgence, microservices, Spring Boot, Angular, Kafka, Kubernetes, Kanban, observabilité.

Acknowledgements	i
Abstract	ii
Résumé	iii
List of Acronyms	xi
General Introduction	1
1 Project Context and Host Organization	3
1.1 Keyrus Group	3
1.2 Keyrus MEA	4
1.3 Internship Mission	5
1.4 Problem Statement	5
1.5 Study of Existing Solutions	6
1.5.1 General Coordination Approaches	6
1.5.2 Representative Digital Platforms	6
1.6 Proposed Solution: Qatra	8
1.7 Objectives and Constraints	8
1.8 Project Stakeholders	9
2 State of the Art and Technological Choices	10
2.1 Digital Blood-Donation Platforms	10
2.2 Architectural Style	10
2.2.1 Monolith and Microservices	10
2.2.2 Hexagonal Architecture	11

2.3	Event-Driven Communication	11
2.4	Backend Framework: Java and Spring Boot	12
2.5	Frontend Framework: Angular	12
2.6	Persistence and Messaging Data	12
2.6.1	PostgreSQL	12
2.6.2	Redis	12
2.6.3	Kafka	13
2.7	Security and Privacy	13
2.8	Containers and Kubernetes	13
2.9	CI/CD and Observability	13
2.10	Technology Decision Summary	15
3	Requirements Analysis and Kanban Planning	16
3.1	System Actors	16
3.2	Functional Requirements	17
3.2.1	Identity and Profile	17
3.2.2	Donor Lifecycle	17
3.2.3	Centers and Appointments	17
3.2.4	Emergencies and Notifications	17
3.2.5	Administration and Governance	17
3.3	Non-Functional Requirements	17
3.4	Product Backlog	19
3.5	Kanban Method	20
3.6	Delivery Increments	21
3.7	Definition of Done and Traceability	21
4	System Architecture and Detailed Design	23
4.1	Global Architecture	23
4.2	Global Use Case Diagram	25
4.3	Global Class Diagram	26
4.4	Bounded Contexts and Modules	27
4.5	Persistence Design	28
4.6	API and DTO Boundaries	28
4.7	Authentication and Authorization	29
4.8	Appointment Consistency	29
4.9	Physical Deployment Architecture	30

4.10	Cross-Cutting Concerns	32
5	Functional Implementation Through Kanban Increments	34
5.1	Increment 1: Foundation, Identity, and Centers	34
5.1.1	Objective and Scope	34
5.1.2	Design Decisions	34
5.1.3	Acceptance Scenarios	37
5.2	Increment 2: Donor Profiles and Appointments	38
5.2.1	Objective and Scope	38
5.2.2	Workflow	38
5.2.3	Privacy Considerations	42
5.3	Increment 3: Staff Operations and Donation Completion	42
5.3.1	Objective and Scope	42
5.3.2	Operational Sequence	42
5.3.3	Auditability	45
5.4	Increment 4: Emergency Management	45
5.4.1	Objective and Scope	45
5.5	Increment 5: Matching and Notifications	48
5.5.1	Objective and Scope	48
5.6	Increment 6: Analytics, Administration, and Operations	51
5.7	Increment Review	53
6	Emergency Matching and Notification Intelligence	54
6.1	Matching Inputs	54
6.2	Blood Compatibility	55
6.3	Eligibility and Availability	55
6.4	Geographical Distance	55
6.5	Ranking and Tiered Expansion	55
6.6	Event Publication and Consistency	57
6.7	Notification Channels	57
6.8	Delivery State, Retry, and Idempotency	57
6.9	Real-Time Updates	57
6.10	Reliability Score and Fairness	58
6.11	GDPR and Audit	58
6.12	Failure Scenarios	58
7	Industrialization, Testing, and Operational Readiness	60

7.1	Repository and Delivery Organization	60
7.2	Continuous Integration	60
7.3	Continuous Deployment	61
7.4	Container and Configuration Management	61
7.5	k3s Deployment	62
7.6	Observability Architecture	63
7.7	Automated Test Inventory	64
7.7.1	Reproducibility Verification Performed for This Report	64
7.8	Recommended Test Strategy	65
7.8.1	Domain and Unit Tests	65
7.8.2	Persistence and Integration Tests	65
7.8.3	API and Security Tests	66
7.8.4	Frontend Tests	66
7.8.5	Operational Tests	66
7.9	Security and Production Readiness	66
7.10	Problems Encountered and Corrective Directions	67
7.10.1	Build Reproducibility	67
7.10.2	Frontend Dependency Setup	67
7.10.3	Environment Completeness	67
7.10.4	Evidence Gaps	67
7.11	Improvement Roadmap	68
	General Conclusion	69
	Bibliography	70

LIST OF FIGURES

1.1	Keyrus AI identity	4
1.2	Keyrus organizational structure	4
2.1	Qatra technology stack and engineering concerns	14
3.1	Global Qatra use-case overview	18
3.2	Kanban process	21
3.3	Functional dependency roadmap for Kanban increments	22
4.1	Verified logical architecture and deployable service boundaries of Qatra	24
4.2	Global Qatra use-case diagram	25
4.3	Global Qatra class diagram	26
4.4	Appointment module architecture	28
4.5	Authentication and protected request sequence	29
4.6	JWT security and role-based authorization flow	30
4.7	Appointment booking, check-in, screening, and completion sequence	30
4.8	Appointment booking and donation activity diagram	31
4.9	Physical k3s deployment in qatra namespace	32
5.1	Login and account verification interfaces	35
5.2	Donation-center discovery interfaces	36
5.3	Center registration and approval interfaces	37
5.4	Donor onboarding workflow	39
5.5	Notification-preference configuration	40
5.6	Appointment center and slot selection	41
5.7	Appointment booking confirmation	42

5.8	Center staff schedule and check-in interfaces	43
5.9	QR check-in and staff processing queue	44
5.10	Health screening form	45
5.11	Emergency creation, candidate matching, and response sequence	46
5.12	Emergency management and request creation	47
5.13	Emergency progress and donor-response workflow	48
5.14	Kafka event and notification delivery flow	49
5.15	Donor emergency alert and response interfaces	50
5.16	Notification access and emergency-status interfaces	51
5.17	Super-administration analytics dashboards	52
5.18	Super-administration audit log interface	52
6.1	Emergency matching and notification activity diagram	56
7.1	Qatra CI/CD pipeline from change to k3s rollout	61
7.2	Metrics, logs, and traces in the Qatra environment	63

LIST OF TABLES

1.1	Comparison of blood-donation coordination approaches	6
1.2	Comparison of representative blood-donation platforms	7
1.3	Qatra system context and principal stakeholders	8
1.4	Qatra stakeholders and expectations	9
2.1	Architectural style comparison	11
2.2	Summary of selected technologies and trade-offs	15
3.1	Qatra actors and authorization boundaries	16
3.2	Principal non-functional requirements	19
3.3	Qatra product backlog by epic	19
3.4	Kanban-versus-Scrum methodology justification	20
3.5	Representative requirement-to-implementation traceability	22
4.1	Principal bounded contexts and responsibilities	27
5.1	Kanban increment review and residual risks	53
6.1	Conceptual tiered emergency matching strategy	56
6.2	Emergency matching and notification failure analysis	59
7.1	Observed automated test-file inventory	64
7.2	Reproducible checks and their observed outcomes	65
7.3	Production-readiness foundation and hardening roadmap	67

LIST OF ACRONYMS

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
DTO	Data Transfer Object
GDPR	General Data Protection Regulation
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
KPI	Key Performance Indicator
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études (End-of-Studies Project)
REST	Representational State Transfer
RBAC	Role-Based Access Control
SSE	Server-Sent Events
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
WebSocket	Full-duplex communication protocol over a persistent connection

GENERAL INTRODUCTION

Blood donation is a constant public-health need. Hospitals rely on compatible blood for surgery, trauma, obstetrics, oncology, and chronic care. But the people and systems behind it are spread out: donors decide alone, centers manage limited staff and slots, and emergency coordinators face tight deadlines. When they rely on scattered records, calls, or broad appeals, time is lost and response is hard to audit.

Digital coordination can reduce this fragmentation, but the problem is more complex than publishing a list of centers. A useful platform must manage identity and consent, medical eligibility, geographical proximity, appointment capacity, staff workflows, notification preferences, emergency deadlines, and data protection. It must also remain responsive when many eligible donors are evaluated or notified. These needs naturally lead to questions of modularity, asynchronous communication, security, and operational visibility.

Qatra was developed as an end-of-studies project at Keyrus MEA to address this challenge. It is a web platform that connects donors, center staff, center administrators, and super administrators. Its functional scope covers donor onboarding, blood-type and health information, location and availability, center discovery, appointment scheduling, screening and completion, donation history, emergency blood requests, tiered donor matching, notification delivery, audit records, and administrative analytics.

The technical solution combines an Angular 20 frontend with two Java 21 and Spring Boot services. The donation service contains the principal identity, donor, center, appointment, emergency, and analytics capabilities. The notification service handles notification-oriented responsibilities and real-time communication. PostgreSQL provides relational persistence, Kafka supports event exchange, and Redis participates in the operational environment. Docker Compose supports local orchestration while k3s manifests describe a production-like deployment with Nginx, monitoring, logging, and tracing components.

The work was organized with Kanban. This choice supported a continuous flow of analysis, implementation, verification, and documentation without forcing artificial sprint boundaries. Work items were grouped into coherent delivery increments so that domain foundations preceded workflows that depended on them.

This report is organized into seven chapters. The first presents Keyrus MEA, the internship mission, and the project problem. The second positions the technological choices. The third formalizes actors, requirements, and Kanban planning. The fourth describes architecture and detailed design. The fifth follows the functional implementation through delivery increments. The sixth focuses on emergency matching and notification intelligence. The seventh examines industrialization, testing, observability, limitations, and operational readiness. A general conclusion summarizes the achieved result and identifies realistic extensions.

CHAPTER 1

PROJECT CONTEXT AND HOST ORGANIZATION

Introduction

This chapter establishes the organizational and functional context of Qatra. It presents the host organization, clarifies the internship mission, analyzes the coordination problem, and identifies the stakeholders whose expectations shaped the platform.

1.1 Keyrus Group

Keyrus is an international consulting and technology group whose activities include data intelligence, digital experience, and business transformation. Its teams combine advisory work, systems integration, data engineering, software delivery, and operational support. This multidisciplinary positioning is aligned with Qatra because the project connects a social domain problem with software architecture, user experience, data consistency, DevOps, and observability concerns [1].

The group works through regional entities that combine local delivery with access to wider expertise. Engineering projects are approached as business transformations rather than isolated coding exercises: requirements, architecture, quality, deployment, and maintainability are considered throughout the delivery lifecycle.



Figure 1.1: Keyrus AI identity and “Architect of intelligence” signature

1.2 Keyrus MEA

Keyrus MEA serves projects in the Middle East and Africa and collaborates with the wider Keyrus ecosystem. The internship environment exposed the project to enterprise delivery practices: explicit responsibilities, incremental planning, architectural justification, source control, automated workflows, and operational documentation. Qatra was developed as a coherent engineering product within this context, not as a disconnected prototype.

Figure 1.2 summarizes the organizational structure used in the reference company presentation. It situates the delivery teams beneath management and business-unit coordination without attributing Qatra to an unsupported internal department.

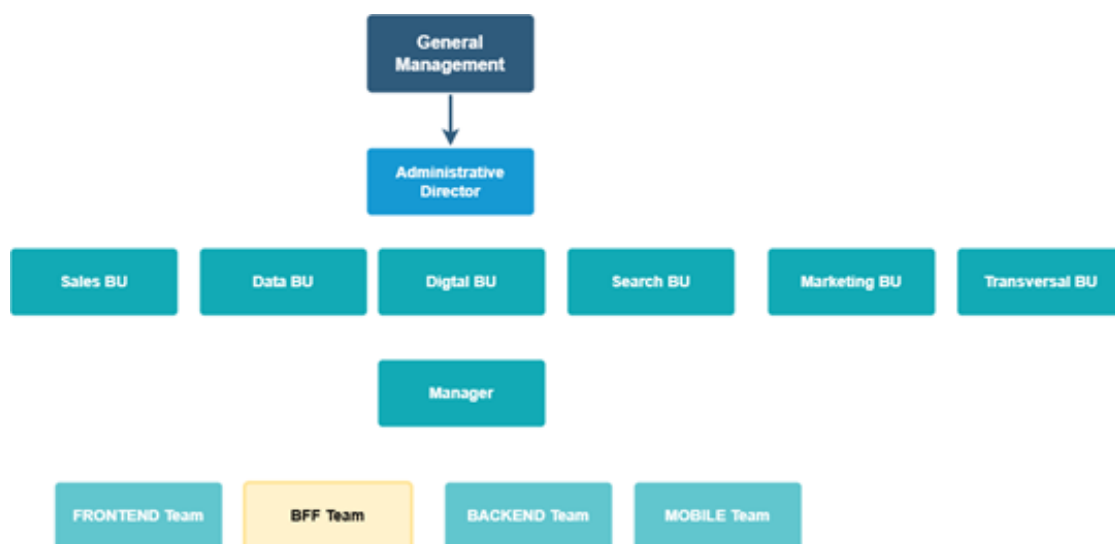


Figure 1.2: Keyrus organizational structure adapted from the host-company material used in the reference report.

The host organization provided a setting in which backend, frontend, infrastructure, and reporting concerns could be treated together. This was important because a blood-donation platform cannot be evaluated only through isolated screens; reliability depends on the consistency of workflows across services and on the ability to observe them after deployment.

1.3 Internship Mission

The internship mission was to design and implement a distributed blood-donation and emergency-response platform. The work included:

- analyzing donor, center, staff, and administrative workflows;
- defining functional and non-functional requirements;
- modeling the domain and its principal state transitions;
- implementing Spring Boot backend services and an Angular frontend;
- integrating persistence, security, asynchronous events, and real-time notifications;
- preparing Docker and k3s deployment resources;
- establishing CI/CD, monitoring, logging, and tracing configurations;
- documenting design decisions, validation evidence, and known limitations.

The mission required autonomy across several technical layers. It also required scope control: the purpose was to build a defensible platform foundation rather than claim integration with national health systems or operational blood inventories that were not part of the available project environment.

1.4 Problem Statement

Blood-donation coordination involves recurring and urgent needs. Recurring needs require centers to maintain a stable schedule of eligible donors. Emergencies require a focused response based on compatibility, distance, availability, and time. Donor suitability and donation testing remain medical responsibilities governed by qualified professionals and documented guidance, including the World Health Organization's donor-selection and blood-screening recommendations [21, 22]. Traditional coordination mechanisms present several limitations:

- donor records and availability may be fragmented or outdated;
- broad appeals notify people who are incompatible, unavailable, or too distant;
- appointment capacity and operating hours are not always visible to donors;
- staff must repeat administrative checks during check-in and screening;
- emergency progress is difficult to follow in real time;
- donors receive little feedback about eligibility, history, and impact;

- operational actions may lack a consistent audit trail.

The central question is therefore: *how can a secure and maintainable digital platform coordinate donors, centers, appointments, emergencies, and notifications while keeping workflows traceable and responsive?*

1.5 Study of Existing Solutions

1.5.1 General Coordination Approaches

Existing approaches can be grouped into manual coordination, institutional portals, campaign-oriented applications, and integrated digital platforms. Table 1.1 summarizes their strengths and limits.

Approach	Strength	Typical limitation
Telephone and spreadsheets	Familiar and inexpensive to start	Fragmented data, manual filtering, weak traceability, limited scalability
Institutional information portal	Authoritative center and campaign information	Often informational rather than transactional; limited personalization
Social-media emergency appeal	Rapid reach to a broad audience	No systematic eligibility, compatibility, distance, or response tracking
Standalone appointment application	Improves scheduling and reminders	May not cover emergencies, staff workflows, and cross-center administration
Integrated coordination platform	Unified donor, center, appointment, and emergency workflows	Requires stronger architecture, security, operations, and governance

Table 1.1: Comparison of blood-donation coordination approaches

The critique supports an integrated approach but does not imply that every responsibility should be placed in one software component. Functional integration can coexist with technical modularity. Qatra therefore presents a unified user experience while separating principal backend responsibilities and using events for cross-service communication.

1.5.2 Representative Digital Platforms

To position Qatra against real services, this study examines five publicly documented platforms from Tunisia and other countries. Table 1.2 compares only capabilities described by their publishers. A feature that is not mentioned in the consulted material is treated as outside the published comparison, not as proof that the platform does not provide it.

Platform	Published capabilities	Position relative to Qatra's scope
CNTS Tunisia appointment platform	Online appointment booking for the national center in Tunis and the regional center in Sfax, intended to reduce waiting time, with extension to other regional centers announced [18].	Provides the most relevant local institutional reference. Qatra extends the study beyond appointment booking to donor profiles, center operations, emergency matching, notifications, analytics, and platform governance.
American Red Cross Blood Donor App	Finds nearby drives and centers, schedules and manages appointments, supports RapidPass, displays donation history and health information, and tracks a donation's journey [19].	Demonstrates a mature donor-centered experience. Qatra studies a broader multi-role workflow that also includes center staff, administrators, emergency coordination, and auditable operational actions.
Facebook Blood Donations	Allows users to register for nearby blood-donation notifications and discover requests, blood banks, and donation events; Meta reported availability in Tunisia in 2021 [20].	Optimized for broad community reach and awareness. Qatra's design instead emphasizes verified platform roles, eligibility and compatibility checks, controlled emergency states, targeted delivery, and auditability.

Table 1.2: Comparison of representative blood-donation platforms

The comparison reveals two complementary tendencies. Institutional applications provide reliable appointment and donor-account services, while social platforms provide large-scale awareness and rapid dissemination. Qatra brings these concerns into one controlled domain model: appointments remain linked to center capacity, emergency alerts are filtered before dispatch, staff actions follow explicit states, and administrative operations are recorded. This positioning is a design objective of the project; it is not a claim of superior clinical or production performance.

1.6 Proposed Solution: Qatra

Qatra is a role-aware web platform centered on the donation lifecycle. A donor creates and verifies an account, completes profile and health information, sets location and availability, discovers centers, reserves a slot, completes screening, and receives a recorded donation outcome. Center staff manage daily operational tasks and authorized emergency workflows. Center administrators manage center information, slots, staff, activity, reports, and urgent requests within their scope. Super administrators govern users, center approvals, roles, audit records, and sensitive requests. The system filters and ranks eligible donors and dispatches notifications.

The platform context is summarized in Table 1.3. It identifies the principal actors, the Qatra boundary they use, and the information exchanged with the platform.

Actor	Qatra boundary	Principal interaction
Donor	Donor and appointment workflows	Profile, health information, eligibility, availability, center discovery, booking, alerts, and responses
Center staff	Donation operations	Daily schedule, donor check-in, health screening, completion, and emergency coordination
Center administrator	Center governance	Center information, operating hours, slots, staff, activity, analytics, and reports
Super administrator	Platform governance	Users, roles, center approvals, restrictions, GDPR requests, dashboards, and audit records

Table 1.3: Qatra system context and principal stakeholders

1.7 Objectives and Constraints

The principal objectives are to centralize donor and center workflows, reduce irrelevant emergency notifications, improve appointment coordination, preserve an audit trail, and provide an architecture that can evolve. The principal constraints are the sensitivity of health-related data, the need for role separation, asynchronous failure handling, geographical calculations, consistency across service boundaries, and the absence of direct integration with operational hospital systems in the project scope.

Success is therefore evaluated through coverage and coherence of implemented workflows, source-backed architecture, automated test presence, and operational configuration. It is not evaluated through unverified clinical impact or production usage figures.

1.8 Project Stakeholders

Stakeholder	Responsibility	Principal expectations
Donor	Maintain profile, availability, appointments, and emergency responses	Simple onboarding, privacy, clear eligibility, useful reminders, accessible center information
Center staff	Execute check-in, screening, and appointment completion	Accurate daily schedule, fast access to authorized records, auditable actions
Center administrator	Operate a donation center and its team	Configurable slots, staff management, reports, activity visibility
Super administrator	Govern the platform	User and role control, center approval, audit and GDPR workflows
Company supervisor	Guide professional delivery	Visible progress, justified design, maintainable implementation
Academic supervisor	Guide academic quality	Traceable requirements, critical analysis, coherent validation
DevOps operator	Deploy and observe the platform	Repeatable releases, health information, logs, metrics, and rollback options

Table 1.4: Qatra stakeholders and expectations

Conclusion

This chapter defined the professional context, the coordination problem, the stakeholders, and the boundaries of Qatra. The next chapter compares the architectural and technological alternatives that support these requirements.

CHAPTER 2

STATE OF THE ART AND TECHNOLOGICAL CHOICES

Introduction

Selecting technologies for Qatra required more than assembling popular frameworks. Each choice had to support domain modularity, secure web interaction, reliable persistence, asynchronous coordination, and observable deployment. This chapter positions the selected stack against reasonable alternatives.

2.1 Digital Blood-Donation Platforms

A modern blood-donation platform combines three categories of capability. The first is donor engagement: registration, eligibility guidance, location, appointment booking, reminders, and donation history. The second is center operations: capacity, staff tasks, screening, completion, and reports. The third is urgent coordination: emergency creation, compatibility filtering, geographical matching, notification, acceptance, and resolution.

These capabilities share data but have different operational rhythms. Appointment planning is predictable; emergency matching is time-sensitive and bursty. Administrative reporting is read-oriented; screening and emergency response change sensitive state. Architecture must reflect those differences while preserving a coherent user journey.

2.2 Architectural Style

2.2.1 Monolith and Microservices

A modular monolith offers simple deployment and local transactions. It is often appropriate for an early product. Microservices provide stronger deployment and scaling boundaries but introduce distributed communication, observability, and consistency costs [2]. Qatra

adopts a pragmatic microservices-oriented structure: a principal donation service contains closely related domain modules, while notification responsibilities are separated. This avoids premature fragmentation into many small services while giving asynchronous delivery a distinct operational boundary.

Criterion	Layered monolith	Fine-grained microservices	Qatra choice
Deployment simplicity	High	Low	Moderate: two backend services
Local consistency	Strong	Requires coordination	Domain writes remain local where possible
Independent scaling	Limited	High	Notification processing can evolve separately
Operational complexity	Low	High	Controlled through containers and observability
Domain modularity	Depends on discipline	Enforced by boundaries	Hexagonal modules inside the donation service

Table 2.1: Architectural style comparison

2.2.2 Hexagonal Architecture

The donation service organizes many features around domain models, inbound ports, outbound ports, application services, and infrastructure adapters. This reflects hexagonal architecture: business rules depend on abstractions, while persistence and web concerns implement external adapters [3]. The benefit is visible in modules such as appointment, center, donor, emergency, and analytics, where domain vocabulary is not reduced to controller or database terminology.

2.3 Event-Driven Communication

Synchronous HTTP is appropriate when a caller needs an immediate response. Notification delivery is different: once a domain event is accepted, email or real-time delivery may take longer or temporarily fail. Kafka supports durable asynchronous exchange and decouples the producer's transaction from downstream processing [4]. The trade-off is eventual consistency and the need for retry, idempotency, correlation, and monitoring.

Qatra uses Kafka in the infrastructure and service configuration as the event backbone between donation-oriented actions and notification processing. The design chapter explains how this boundary is represented without claiming stronger delivery guarantees than the code and configuration demonstrate.

2.4 Backend Framework: Java and Spring Boot

Java 21 provides a current long-term-support runtime with a mature ecosystem. Spring Boot supplies web, validation, security, persistence, actuator, mail, WebSocket, and testing integration [5]. The project's Maven descriptors target Java 21 and Spring Boot 4.0.6.

Alternatives such as Node.js or .NET could support the same domain. Spring Boot was retained because the project benefits from typed domain models, mature JPA integration, declarative validation, filter-based security, and consistent operational endpoints. Its main costs are framework complexity and a larger runtime footprint than minimal services.

2.5 Frontend Framework: Angular

Angular provides a structured application framework with dependency injection, routing, reactive forms, HTTP integration, and build tooling [6]. Qatra uses Angular 20, RxJS, NgRx Signals, PrimeNG, Zod, Chart.js, MapLibre, and an HTML5 QR-code library. These dependencies align with role-based dashboards, typed API integration, validation, maps, analytics, and appointment check-in.

Angular's convention-rich structure is useful for a broad feature set, but it requires disciplined separation of core concerns, shared models, feature services, stores, and route guards. The repository follows this organization through feature folders for authentication, centers, donors, emergencies, notifications, and staff.

2.6 Persistence and Messaging Data

2.6.1 PostgreSQL

Relational persistence is appropriate for users, centers, appointments, slots, emergency records, and audit logs because these entities have explicit relationships and state constraints. PostgreSQL provides transactions, indexing, geospatial extensions where needed, and a strong operational ecosystem [7]. JPA repositories map the relational model while adapters protect the domain from direct persistence dependence.

2.6.2 Redis

Redis is included in the deployment environment. It is suitable for short-lived state, caching, counters, and coordination data, but the report does not assign business-critical persistence to Redis unless demonstrated by the source. PostgreSQL remains the authoritative store for relational domain records.

2.6.3 Kafka

Kafka provides append-oriented event streams and consumer groups. Its inclusion is justified where notification workloads should not block domain operations. Schema evolution, duplicate delivery, and dead-letter handling remain architectural responsibilities rather than automatic properties of the broker.

2.7 Security and Privacy

Qatra uses Spring Security and JWT-oriented authentication. Stateless access tokens are suitable for distributed APIs, but they require careful expiration, signature management, authorization checks, and handling of account state [8]. Role-based controls separate donors, staff, center administrators, and super administrators. Health screening, location, and deletion requests are particularly sensitive and must follow least privilege. The OWASP API Security Top 10 provides a practical reference for risks such as broken object-level authorization, broken authentication, unrestricted resource consumption, and security misconfiguration [25].

GDPR-oriented capabilities in the use-case model include account deletion requests and administrative processing. Because health and precise location data require particular care, processing must be purpose-limited, access-controlled, and minimized in accordance with the applicable legal basis and data-subject rights [24]. The platform therefore needs both a user-facing request workflow and an auditable administrative decision. Technical deletion must be reconciled with legal retention requirements; the project models the workflow but does not claim legal certification.

2.8 Containers and Kubernetes

Docker provides reproducible service packaging and local orchestration [9]. The backend repository includes Dockerfiles and a multi-service Compose configuration. The infrastructure repository contains k3s manifests for the namespace, PostgreSQL, Kafka, Redis, both services, Jaeger, Prometheus, Loki, Promtail, Grafana, ingress, authentication middleware, Kubernetes Dashboard, and pgAdmin.

k3s provides a lightweight Kubernetes distribution suitable for a constrained server while retaining Kubernetes deployment, service, ingress, secret, and persistent-volume concepts [10]. This choice adds operational complexity; it is justified by the project's aim to document a production-like distributed environment rather than merely run two local processes.

2.9 CI/CD and Observability

GitHub Actions defines test and deployment workflows [11]. Pull requests trigger a Java 21 Maven test matrix for the donation and notification services. Pushes to the master branch use path filters, build and publish Docker images to GHCR, import them into k3s through SSH, and

restart the relevant deployment. These files demonstrate workflow intent; this report does not invent execution success rates.

Prometheus collects metrics, Grafana visualizes them, Loki centralizes logs, Promtail ships logs, and Jaeger provides distributed tracing [12, 13, 14, 15]. Together they address complementary questions: what is slow or failing, what changed over time, what did a component log, and how did a request traverse services?

Figure 2.1 groups the selected technologies by engineering responsibility and makes the cross-cutting quality concerns explicit.



Figure 2.1: Qatra technology stack and engineering concerns

2.10 Technology Decision Summary

Concern	Selected technology	Reason	Main caution
Backend	Java 21, Spring Boot 4	Typed domain model and integrated ecosystem	Framework and runtime complexity
Frontend	Angular 20	Structured feature-rich SPA	Bundle size and test discipline
Persistence	PostgreSQL, JPA	Relational integrity and transactions	Query and migration governance
Events	Kafka	Decoupled notification processing	Eventual consistency and duplicates
Fast data	Redis	Low-latency operational state	Must not become undocumented authority
Packaging	Docker	Repeatable service images	Image and secret hygiene
Orchestration	k3s	Production-like Kubernetes model	Operational overhead
Delivery	GitHub Actions, GHCR	Repository-integrated automation	Secret and runner security
Metrics	Prometheus, Grafana	Time-series visibility and dashboards	Alert quality and retention
Logs/traces	Loki, Promtail, Jaeger	Cross-component diagnosis	Correlation and storage cost

Table 2.2: Summary of selected technologies and trade-offs

Conclusion

The selected stack is coherent with Qatra’s functional scope and learning objectives. Its strength comes from the interaction between domain modularity, asynchronous communication, typed frontend integration, and operational visibility. The next chapter translates the project context into traceable requirements and a Kanban delivery model.

CHAPTER 3

REQUIREMENTS ANALYSIS AND KANBAN PLANNING

Introduction

This chapter formalizes who uses Qatra, what the system must do, which qualities constrain the solution, and how work was organized as a continuous Kanban flow. The StarUML model supplied with the project is used as the principal use-case source and is cross-checked against controllers, services, frontend routes, and infrastructure files.

3.1 System Actors

The model identifies four human actors: donor, center staff, center administrator, and super administrator.

Actor	Main responsibilities	Data boundary
Donor	Account, profile, health questionnaire, availability, centers, appointments, emergency responses	Own profile and authorized public/transactional data
Center staff	Schedule, check-in, eligibility review, screening, completion, no-show	Operational records for assigned center
Center administrator	Center registration, configuration, staff, slots, analytics, reports	Managed center and its staff/activity
Super administrator	Users, roles, center approval, GDPR, restrictions, global audit/reporting	Platform-wide governance data
System	Tokens, matching, notifications, eligibility, expiration, audit, forecasts	Service accounts and scheduled/event authority

Table 3.1: Qatra actors and authorization boundaries

3.2 Functional Requirements

3.2.1 Identity and Profile

The system shall allow registration by email, email verification, login, logout, password reset, and controlled session management. A donor shall complete personal and health information, set blood type or mark it unknown, define notification preferences, and provide location manually or through GPS. Profile changes must be validated and subject to authorization.

3.2.2 Donor Lifecycle

A donor shall view and update availability, questionnaire answers, location, donation history, eligibility status, next eligible date, reliability score, impact information, and donation certificates. The system shall enforce cooldown rules after a completed donation and expose the result without asking the donor to infer medical dates.

3.2.3 Centers and Appointments

Users shall browse centers and view operating hours. Authorized administrators shall register and update centers, manage staff, configure slots, record operating-hour exceptions, and consult reports. Donors shall book, reschedule, cancel, and review appointments. Staff shall check in donors, record screening, complete appointments, and mark no-shows.

3.2.4 Emergencies and Notifications

Authorized users shall create, track, resolve, and review emergency requests. The system shall scan eligible donors, filter by compatibility and constraints, rank candidates, expand the radius when necessary, dispatch alerts, track delivery, and retry failed notifications. Donors shall view, accept, or decline requests.

3.2.5 Administration and Governance

Super administrators shall search users, change account status, approve or reject centers, assign or revoke roles, process deletion requests, review permanent health restrictions, consult dashboards, and export audit or platform reports. Every state-changing action should generate an attributable audit record.

3.3 Non-Functional Requirements

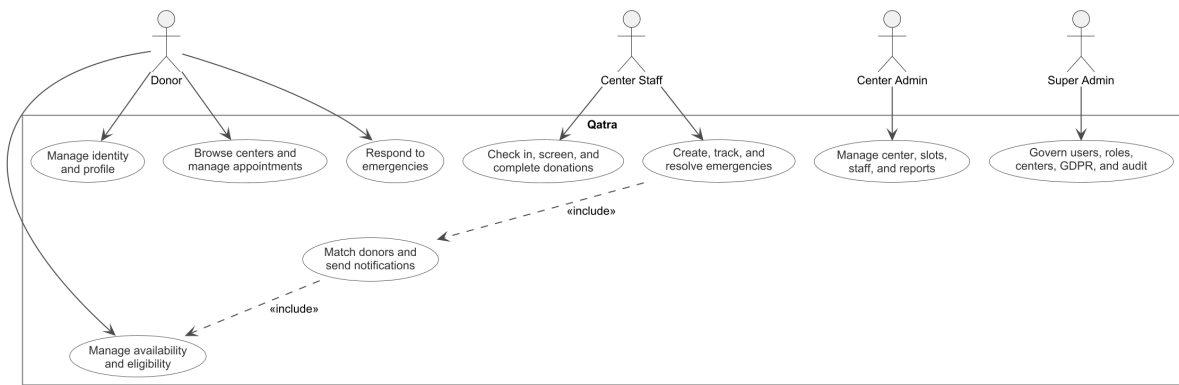


Figure 3.1: Global Qatra use-case overview

Quality	Requirement	Evidence or design response
Security	Authenticate API calls and authorize operations by role and resource scope	Spring Security, JWT dependencies, route guards, role-specific controllers
Privacy	Limit access to health, location, and deletion data	Role separation, GDPR use cases, audit model
Availability	Avoid coupling domain requests to slow notification channels	Kafka-oriented event exchange and separate notification service
Performance	Page list endpoints and avoid unnecessary frontend loading	Pagination utilities, feature routes, asynchronous processing
Consistency	Preserve appointment and slot constraints under concurrent requests	Transactional backend boundaries and explicit slot state
Maintainability	Keep domain rules separate from web and persistence details	Ports, adapters, domain models, DTOs, feature modules
Observability	Expose metrics, logs, and traces for distributed diagnosis	Actuator, Prometheus, Grafana, Loki, Promtail, Jaeger

Quality	Requirement	Evidence or design response
Deployability	Package and update services reproducibly	Dockerfiles, Compose, k3s manifests, GitHub Actions

Table 3.2: Principal non-functional requirements

3.4 Product Backlog

The backlog was organized by user value and dependency. Identity and foundational data precede appointments; appointments and health status precede reliable emergency matching; event exchange precedes real-time notification; operational visibility follows deployable services.

ID	Epic	Representative outcome	Priority
E1	Identity and security	Verified account, login, reset, roles, protected routes	High
E2	Donor onboarding	Health, blood type, location, availability, preferences	High
E3	Center management	Center approval, staff, operating hours, slots	High
E4	Appointment lifecycle	Booking, rescheduling, check-in, screening, completion	High
E5	Donor engagement	History, eligibility, certificates, impact, reliability	Medium
E6	Emergency management	Creation, deadline, progress, resolution, history	High
E7	Matching engine	Compatibility, availability, distance, ranking, escalation	Medium
E8	Notification engine	Alerts, retries, preferences, WebSocket updates	Low
E9	Administration	Users, roles, restrictions, GDPR, audit	Medium
E10	Analytics and reporting	Center and system summaries, exports	Low
E11	Industrialization	CI/CD, containers, k3s deployment	Low
E12	Observability	Metrics, dashboards, logs, traces	Low

Table 3.3: Qatra product backlog by epic

3.5 Kanban Method

Kanban was selected because the work crossed analysis, backend, frontend, infrastructure, and documentation. A continuous pull system allowed an item to move when capacity existed rather than waiting for a sprint boundary [16]. Scrum instead organizes work through time-boxed Sprints, defined accountabilities, and recurring formal events [26]. Table 3.4 explains why Kanban was the more suitable project narrative. The working states were Backlog, Ready, In Progress, Review, Verification, and Done. Blocked work remained visible and did not count as completed.

Criterion	Scrum	Kanban and Qatra fit
Delivery cadence	Work is planned and reviewed within fixed-length Sprints	Work can be pulled and completed continuously across backend, frontend, infrastructure, and report tasks
Change handling	Changes are normally considered without endangering the Sprint Goal	Priorities can be reordered whenever capacity becomes available
Flow control	Sprint Backlog and Sprint Goal guide the time box	Explicit work-in-progress limits expose overload and blocked work
Roles and events	Product Owner, Scrum Master, Developers, and prescribed events	No unsupported claim of historical Scrum roles or ceremonies is required
Project suitability	Strong when a stable team can sustain a regular Sprint rhythm	Better suited to an individual PFE with heterogeneous deliverables and variable task duration

Table 3.4: Kanban-versus-Scrum methodology justification

Work-in-progress limits protect focus. A reasonable individual-project policy is one principal implementation item and one review or documentation item in progress. This policy is described as the project method; no historical throughput figures are invented where no board export exists. Figure 3.2 illustrates the visual pull principle adopted for the project. The detailed Qatra states remain Backlog, Ready, In Progress, Review, Verification, and Done.

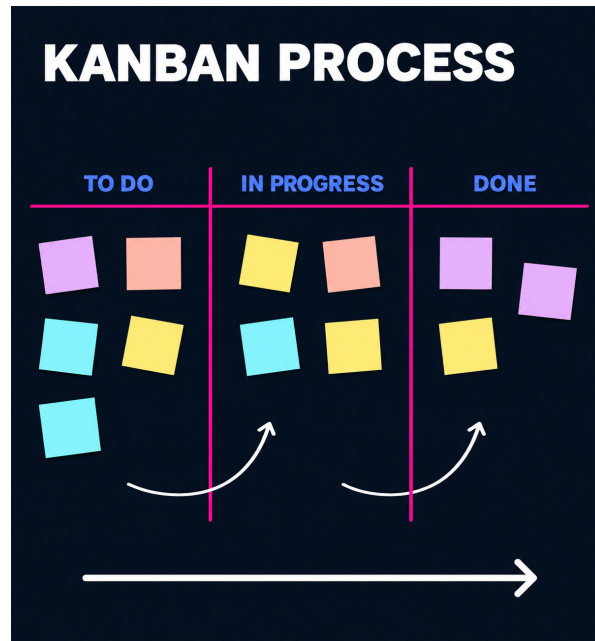


Figure 3.2: Kanban process

3.6 Delivery Increments

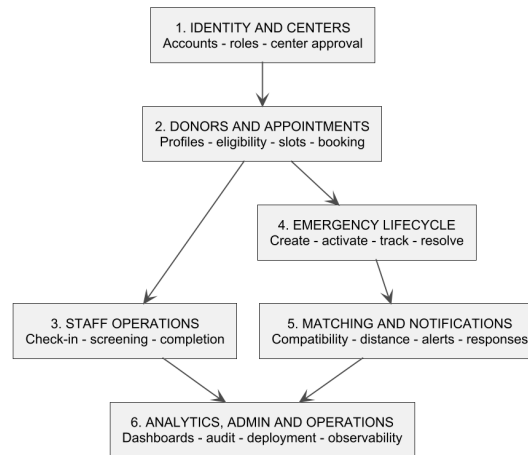
Kanban does not prevent grouping completed work into reportable increments. Six coherent increments are used for traceability:

1. foundation, identity, and center management;
2. donor profiles, center discovery, and appointments;
3. staff operations and donation completion;
4. emergency request lifecycle;
5. matching, notifications, and real-time updates;
6. analytics, administration, deployment, and observability.

3.7 Definition of Done and Traceability

A work item is considered done when its acceptance behavior is implemented, authorization and validation have been considered, affected interfaces are documented, and available automated checks pass in the intended CI environment. Where verification cannot be reproduced locally, the limitation is recorded instead of replacing evidence with an assumption.

Traceability is maintained from actor needs to backlog epics, domain modules, frontend features, tests, and diagrams. Table 3.5 gives representative examples.

**Figure 3.3:** Functional dependency roadmap for Kanban increments

Need	Epic	Backend evidence	Frontend evidence
Book a donation slot	E4	Appointment application/domain/web modules	Slot booking page and appointment models
Operate a center	E3/E4	Center and appointment modules	Center management and staff pages
Respond to an emergency	E6/E7	Emergency and matching responsibilities	Emergency list/detail pages
Receive timely updates	E8	Notification service and event configuration	Notification service, store, bell, and center page
Audit administrative actions	E9	Analytics audit log module	Staff activity and administrative views

Table 3.5: Representative requirement-to-implementation traceability

Conclusion

The requirements and backlog establish a controlled functional scope, while Kanban explains how value was delivered continuously. The next chapter shows how these requirements are mapped to services, modules, data stores, security controls, and deployment nodes.

CHAPTER 4

SYSTEM ARCHITECTURE AND DETAILED DESIGN

Introduction

This chapter maps Qatra’s requirements to deployable components and domain boundaries. It distinguishes logical responsibilities from physical deployment and explains the flows that require particular attention: authentication, appointment concurrency, event publication, authorization, and persistence.

4.1 Global Architecture

The user interacts with an external Angular single-page application. The repository contains exactly two deployable Spring Boot services: the donation service and the notification service. Identity, donor, center, appointment, emergency, matching, analytics, and audit responsibilities are modules inside the donation service rather than independently deployed microservices. Local Docker Compose uses Nginx as its gateway; the k3s environment uses Traefik IngressRoute resources. Domain events pass through Kafka to the notification service, while PostgreSQL and Redis provide persistence and fast operational state.

Figure 4.1 presents the responsibilities and the emphasized emergency path in a single icon-based view.

QATRA LOGICAL ARCHITECTURE

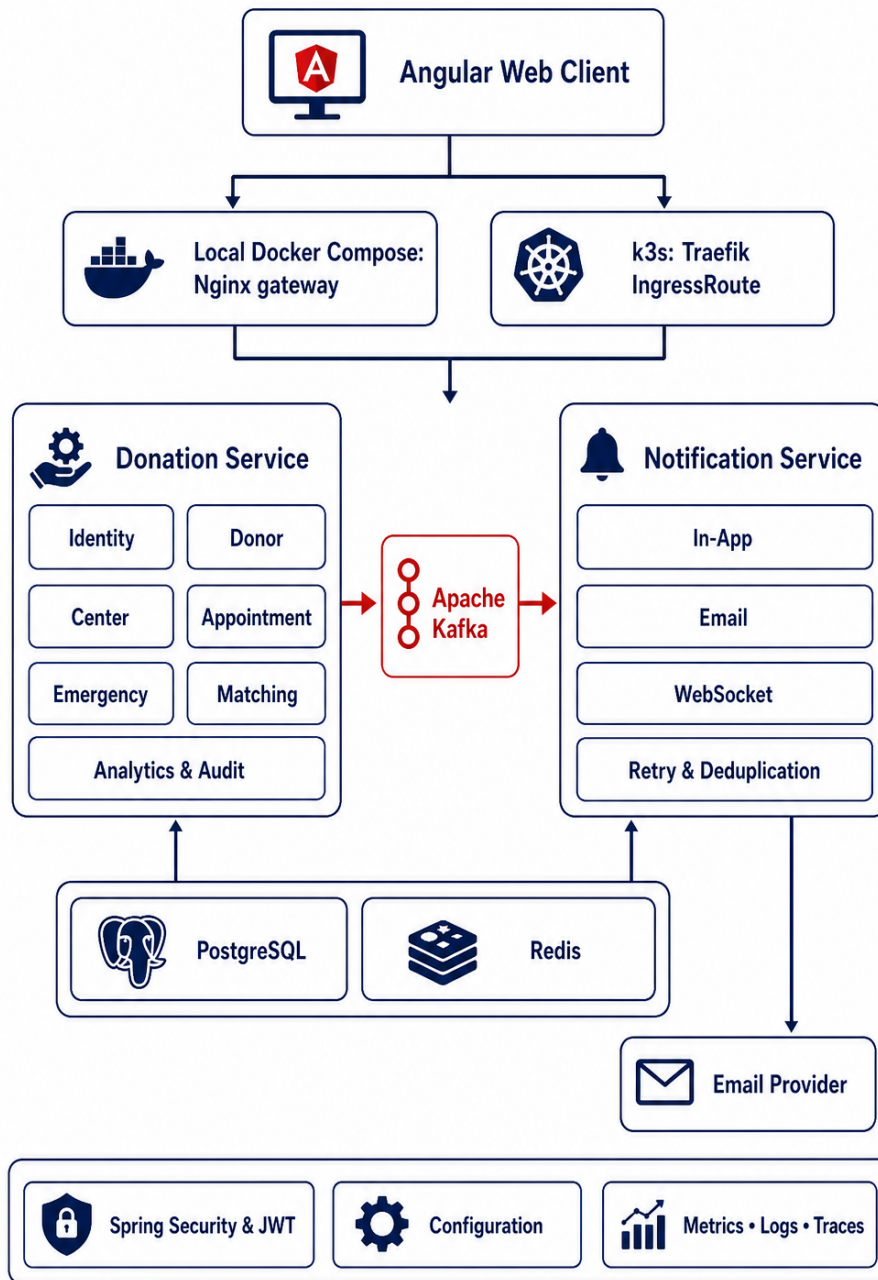


Figure 4.1: Verified logical architecture and deployable service boundaries of Qatra

The architecture avoids direct frontend access to databases or brokers. API boundaries centralize validation, authentication, DTO mapping, and error handling. The notification service is not a second source of truth for donation records; it reacts to identifiers and payloads required for delivery.

4.2 Global Use Case Diagram

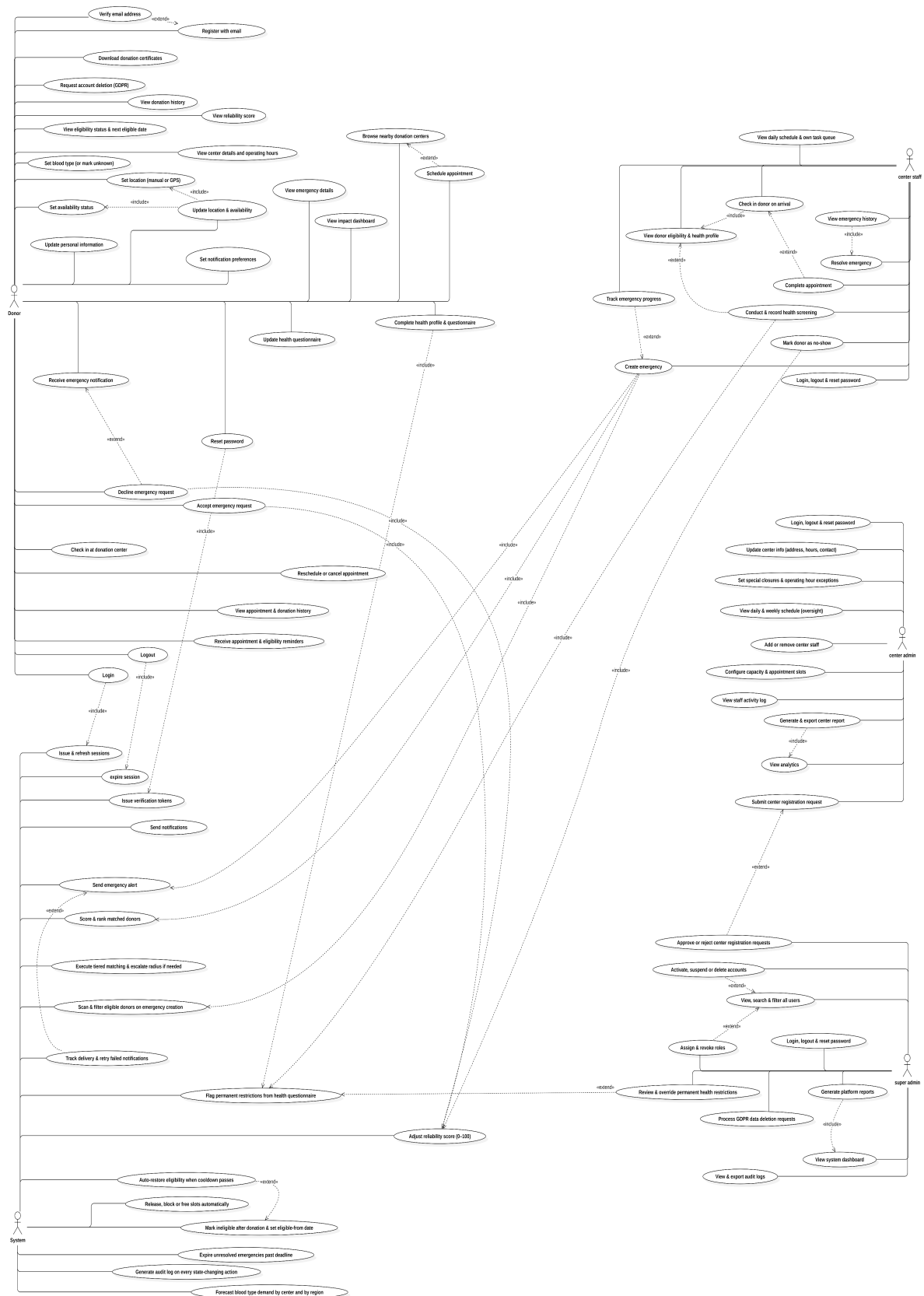


Figure 4.2: Global Qatra use-case diagram

4.3 Global Class Diagram

The global class model in Figure 4.3 consolidates the most important concepts from the implemented packages. It is intentionally a domain overview rather than a dump of every JPA field.

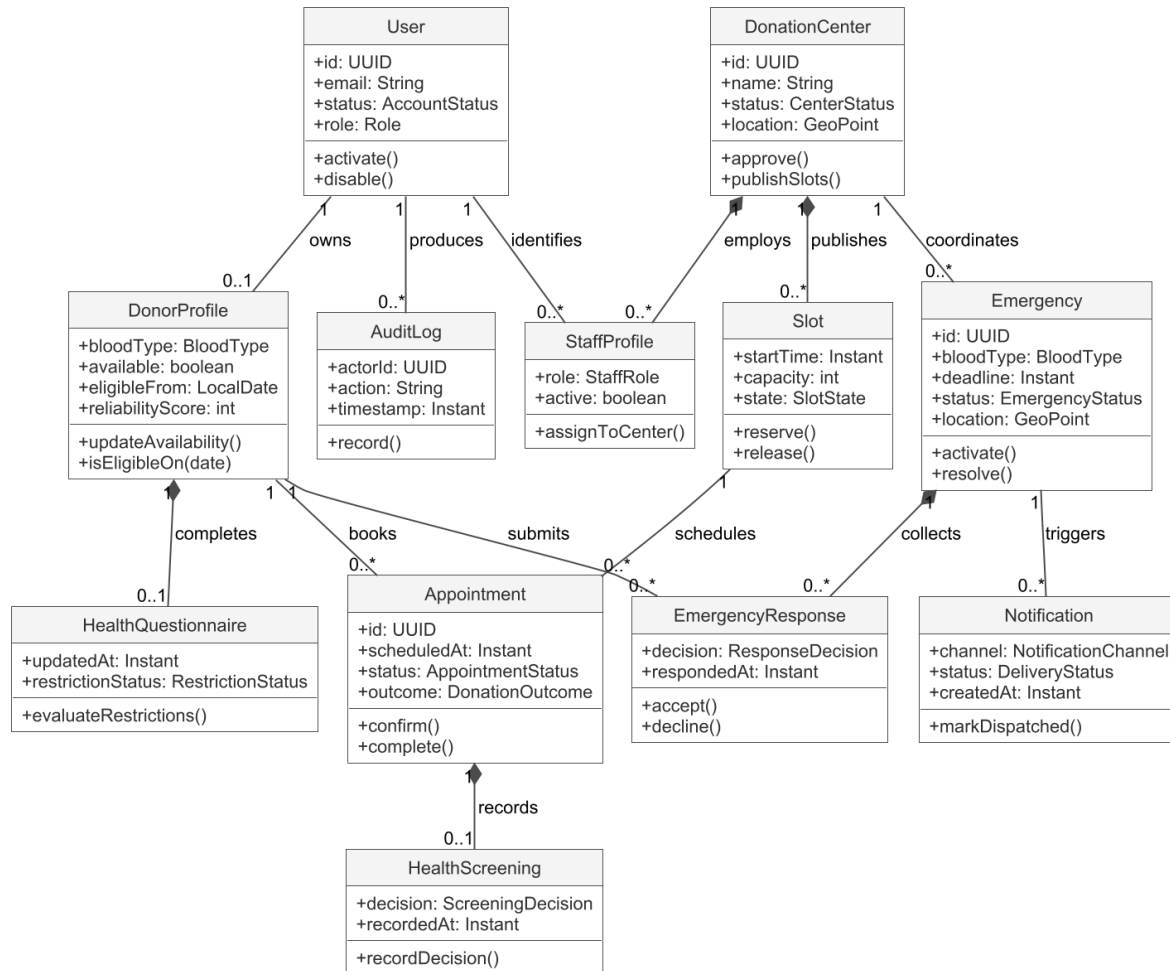


Figure 4.3: Global Qatra class diagram

Important invariants include the following:

- a slot belongs to one center and has finite capacity or state;
- an appointment associates a donor, a center, and a scheduled slot;
- screening precedes a completed donation outcome;
- eligibility depends on health restrictions and the most recent relevant donation state;
- an emergency declares a required blood type, location, deadline, and lifecycle status;
- candidate responses and delivery records must be attributable to the relevant emergency;
- audit records capture actor, action, target, and time for state-changing operations.

4.4 Bounded Contexts and Modules

The donation service contains domain-focused packages rather than a single generic service layer. The principal contexts identified from the repository are shown in Table 4.1.

Context	Responsibilities	Representative concepts
Identity/User	Registration, credentials, verification, roles, sessions	User, role, token, status
Donor	Profile, health, location, availability, eligibility, history	Donor profile, questionnaire, certificate
Center	Center lifecycle, staff, operating hours, slots	Donation center, staff profile, slot, closure
Appointment	Booking, rescheduling, screening, completion	Appointment, screening, outcome
Emergency	Urgent blood request, candidate response, progress	Emergency, blood type, deadline, status
Analytics/Audit	Metrics and state-change traceability	Audit log, center metrics
Notification	Delivery preferences, messages, real-time updates	Notification, channel, delivery state

Table 4.1: Principal bounded contexts and responsibilities

Each context follows a recognizable direction of dependency: web adapters call inbound use cases; application services coordinate domain behavior; repository ports abstract persistence; JPA adapters map domain objects to entities. Proxies provide controlled access when one context needs information owned by another.

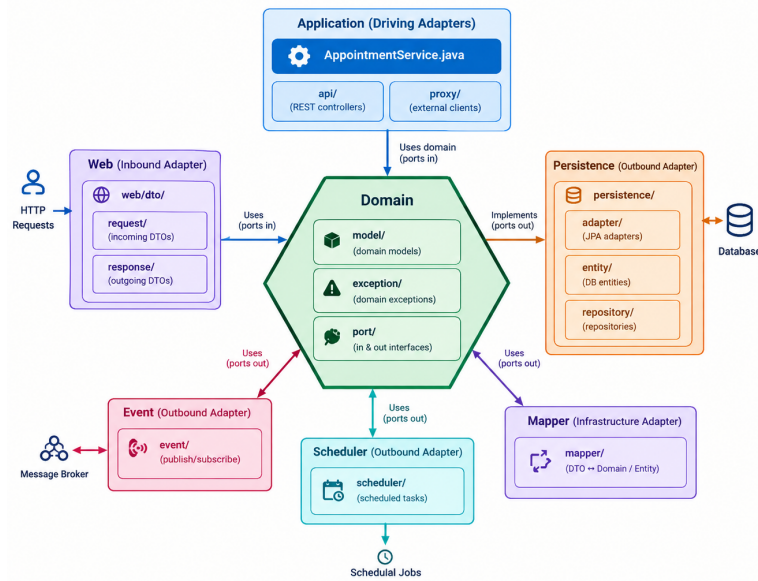


Figure 4.4: Appointment module architecture

4.5 Persistence Design

PostgreSQL stores the relational domain. JPA entities and repositories provide persistence adapters, while mapper classes translate to and from domain models. This separation allows the domain vocabulary to remain stable if storage details change.

Indexes should follow actual query patterns: email and account state for authentication; center and date for schedules; donor and status for appointments; emergency status and deadline for active searches; blood type, availability, eligibility date, and location-related fields for matching; and timestamp, actor, and action for audit queries. Migration scripts and production index evidence should be treated as deployment artifacts; the report does not claim indexes that are not represented in the available schema.

4.6 API and DTO Boundaries

Controllers expose JSON-oriented contracts and delegate to application services. Request DTOs validate input such as appointment creation, screening, center updates, and status changes. Response DTOs prevent persistence entities from becoming public contracts. This boundary is particularly important for health and authorization data: the API should expose only the fields required by the caller's role.

Errors are represented through domain-specific error codes and centralized handling. This makes client behavior predictable and prevents internal stack traces from becoming an API. Pagination is used for lists that can grow, such as centers, appointments, users, emergencies, notifications, and audit records.

4.7 Authentication and Authorization

Authentication verifies credentials and issues signed tokens. Subsequent requests carry a token that is validated before the security context is established. Method or route authorization then checks the role and, where required, resource ownership or center membership.

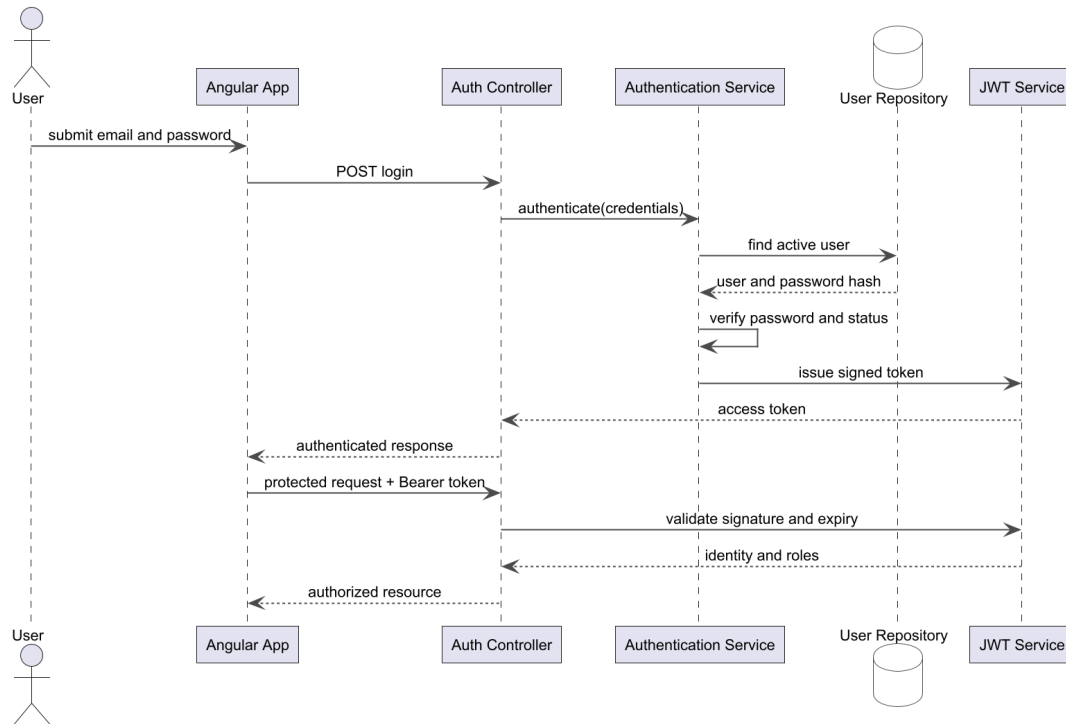


Figure 4.5: Authentication and protected request sequence

Role checks alone are insufficient. A center administrator must not manage another center merely because the role matches. The application layer therefore needs contextual checks using the authenticated user and target entity. Account status, token expiry, and role changes must also be considered when accepting a token.

4.8 Appointment Consistency

Appointment booking combines availability shown in the UI with authoritative server-side validation. Between display and confirmation, another donor may reserve the same capacity. The backend must therefore verify slot state within a transactional boundary and reject conflicts deterministically.

The same workflow is represented as an activity diagram in Figure 4.8, which emphasizes decisions and responsibility changes rather than message timing.

Rescheduling should behave as one business transition: validate the new slot, update or replace the reservation, and leave the previous slot in a consistent state. Check-in and completion are restricted to authorized staff. Completion triggers donation outcome and eligibility consequences rather than only changing a label in the UI.

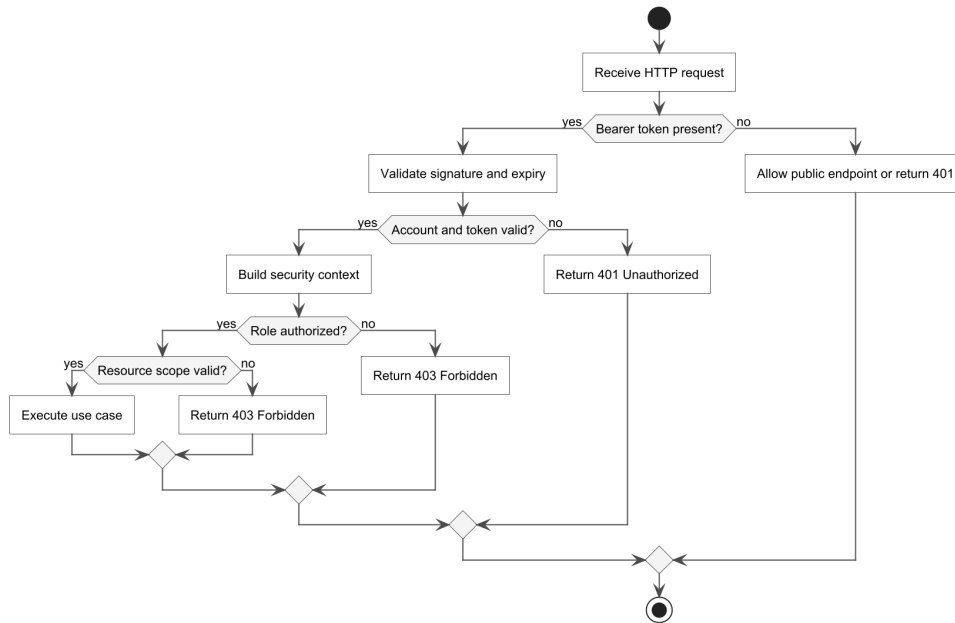


Figure 4.6: JWT security and role-based authorization flow

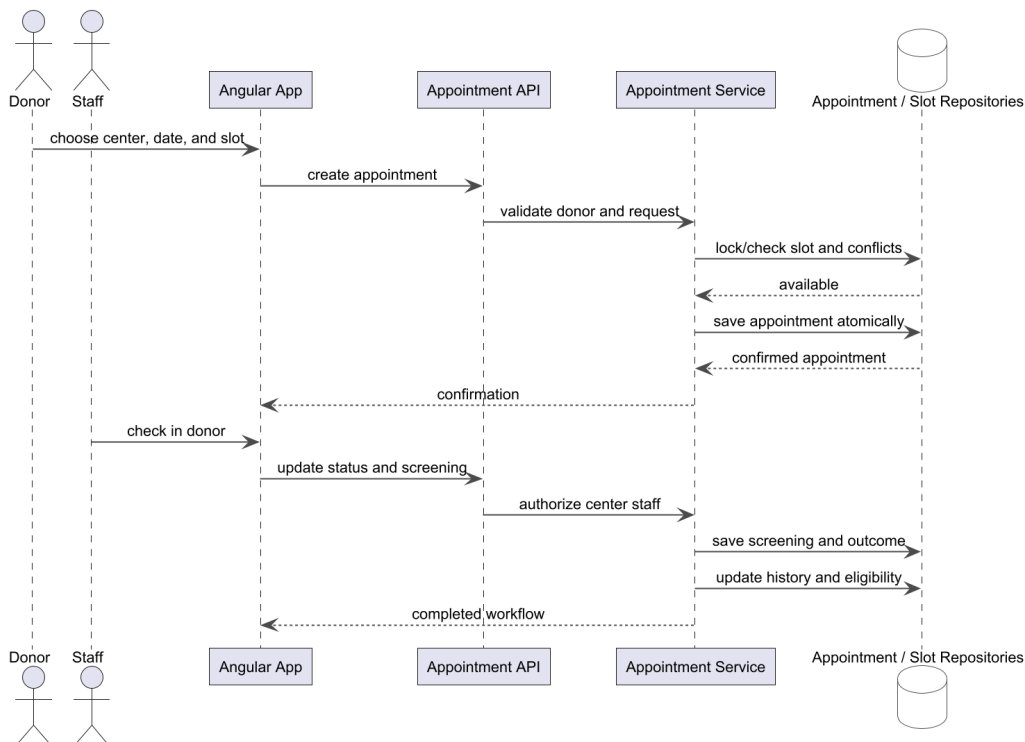
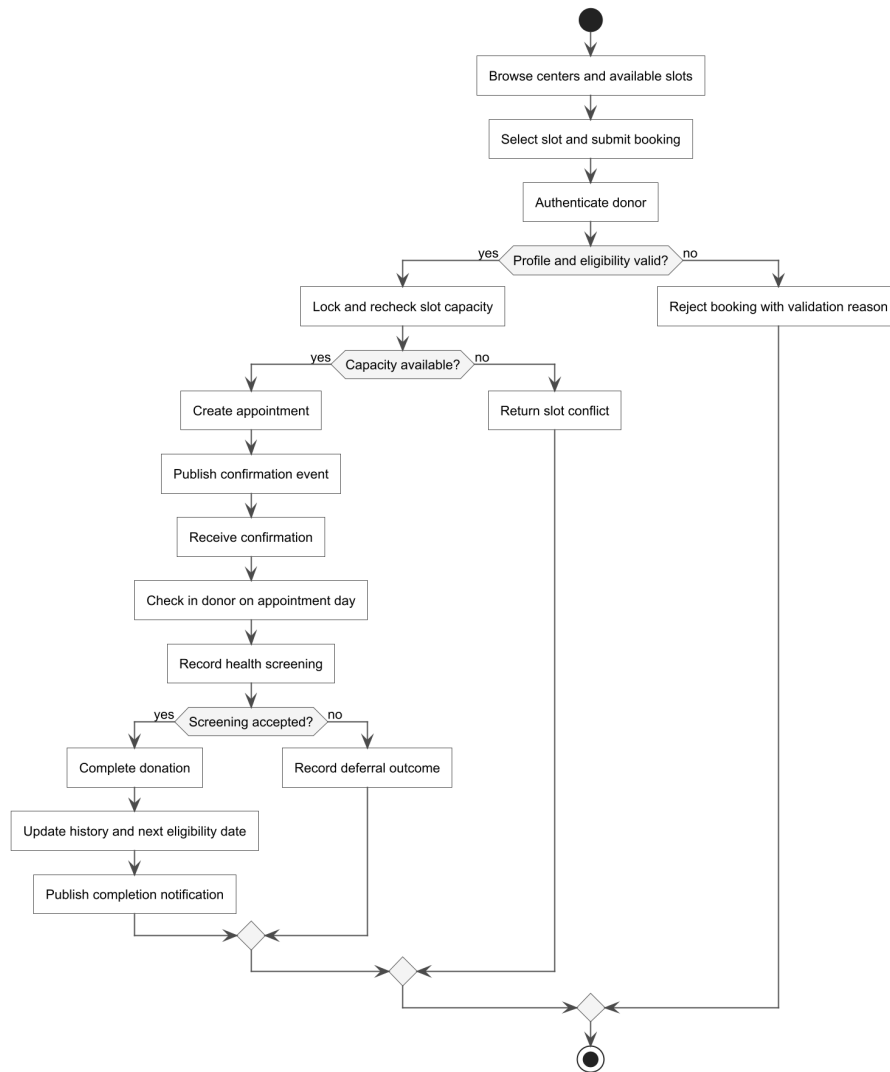


Figure 4.7: Appointment booking, check-in, screening, and completion sequence

4.9 Physical Deployment Architecture

The infrastructure declares one principal Kubernetes namespace, `qatra`. Inside it, the deployable business workloads are the donation service and notification service; PostgreSQL, Kafka, Redis, observability components, and administrative tools support them. Traefik IngressRoute resources expose the donation API, notification API, and notification WebSocket endpoint. Persistent storage is used where state must survive pod replacement.

**Figure 4.8:** Appointment booking and donation activity diagram

QATRA PHYSICAL k3s DEPLOYMENT

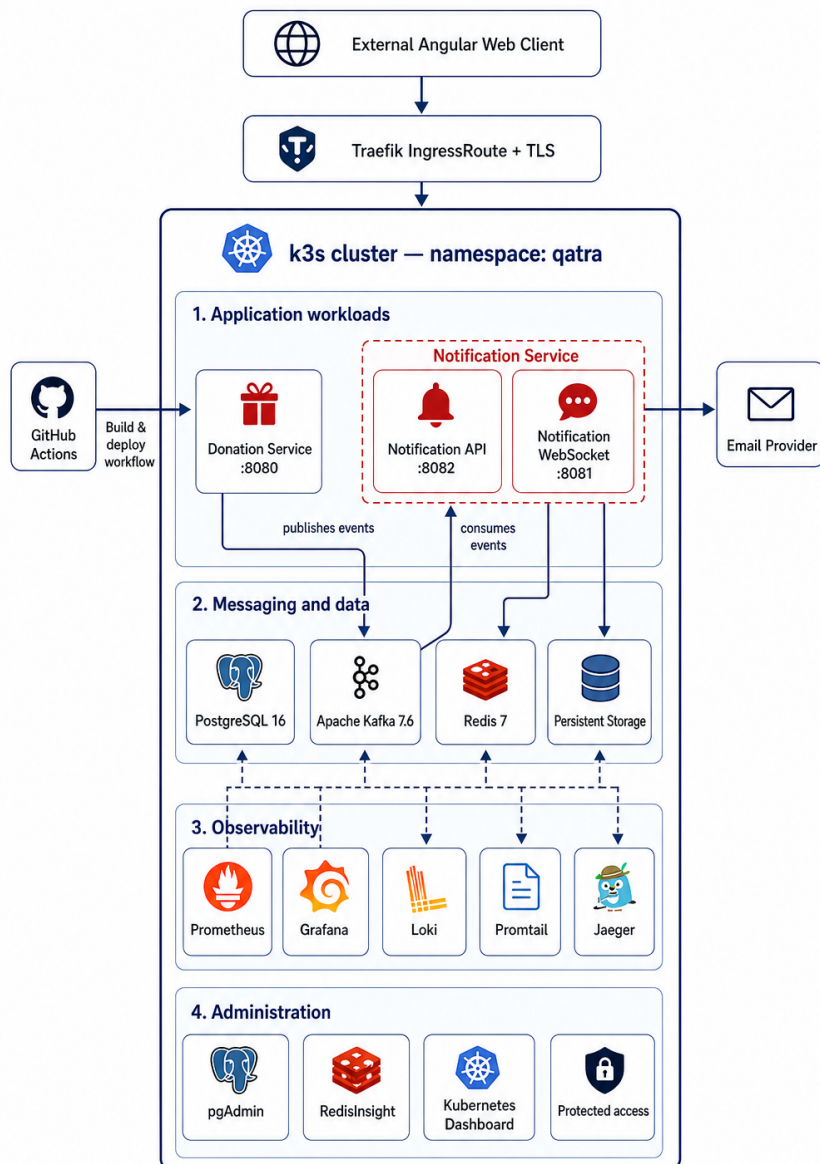


Figure 4.9: Physical k3s deployment in qatra namespace

Traefik provides the k3s entry point, whereas Nginx is limited to the local Compose gateway. Secrets should supply passwords, tokens, and API keys rather than embedding them in images or committed manifests. Readiness and liveness checks should distinguish a process that is running from a service that can accept work.

4.10 Cross-Cutting Concerns

Validation, error mapping, logging, correlation identifiers, audit events, metrics, and tracing apply across contexts. Correlation is particularly important when an emergency command produces a Kafka event and a later notification delivery. Without a shared identifier, logs remain

technically centralized but operationally difficult to interpret.

Conclusion

Qatra's architecture combines domain-oriented modules with a deliberately small service boundary, asynchronous notification flow, and production-like infrastructure. The following chapter uses the Kanban increments to connect this design to concrete user-facing capabilities.

CHAPTER 5

FUNCTIONAL IMPLEMENTATION THROUGH KANBAN INCREMENTS

Introduction

This chapter presents implementation as a sequence of value-oriented Kanban increments. The increments are reporting groupings, not time-boxed sprints. Each one combines requirements, backend behavior, frontend interaction, and validation considerations.

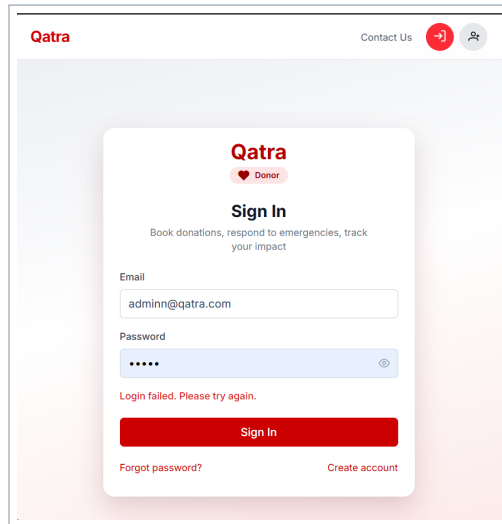
5.1 Increment 1: Foundation, Identity, and Centers

5.1.1 Objective and Scope

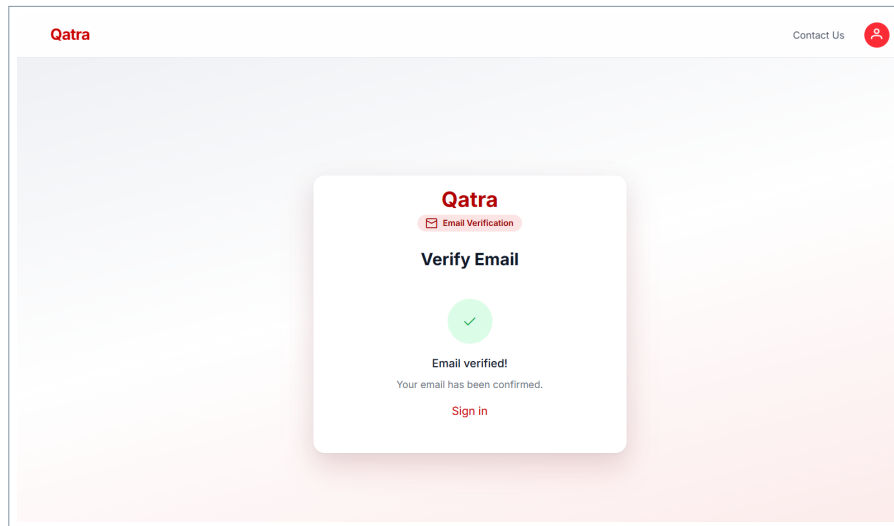
The first increment established the application skeleton, user identity, protected routing, and donation-center foundation. These capabilities were prerequisites for all role-aware workflows. On the backend, authentication and user concerns integrate Spring Security, token handling, validation, and persistence. Center modules provide command and query use cases, mappers, repository adapters, slot management, staff profiles, operating hours, closures, and administrative controllers. On the frontend, authentication pages cover registration, login, verification, forgotten password, and reset. Center routes and services expose listing, detail, creation, and management views.

5.1.2 Design Decisions

Email verification separates account creation from trusted access. Password reset uses short-lived proof rather than exposing or transmitting an existing password. Center registration and approval are separate because submission by a center actor does not imply platform authorization. Staff membership is modeled explicitly so that access can be limited to a center.

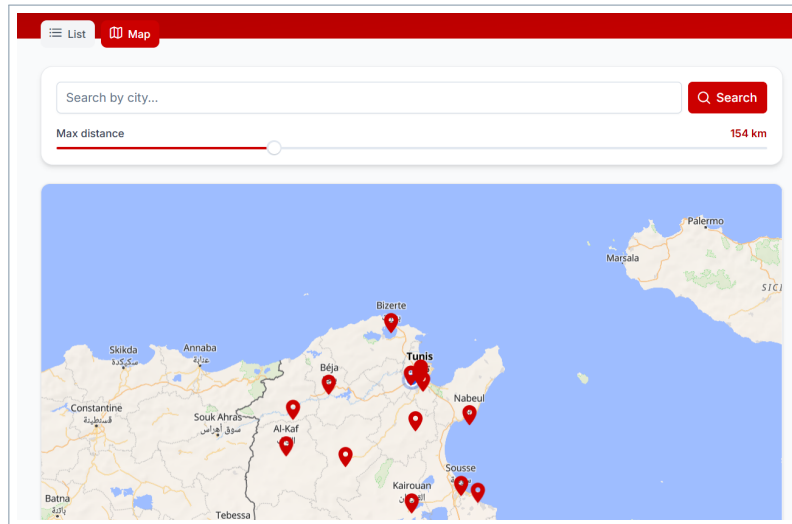


(a) Donor sign-in

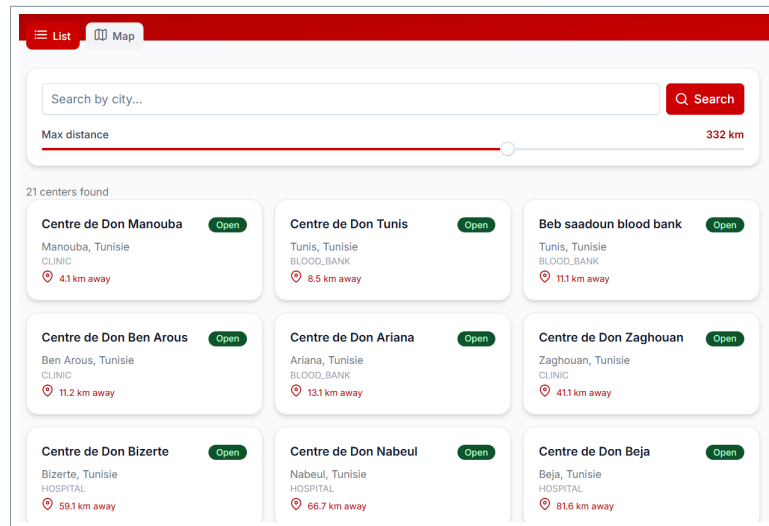


(b) Successful email verification

Figure 5.1: Login and account verification interfaces

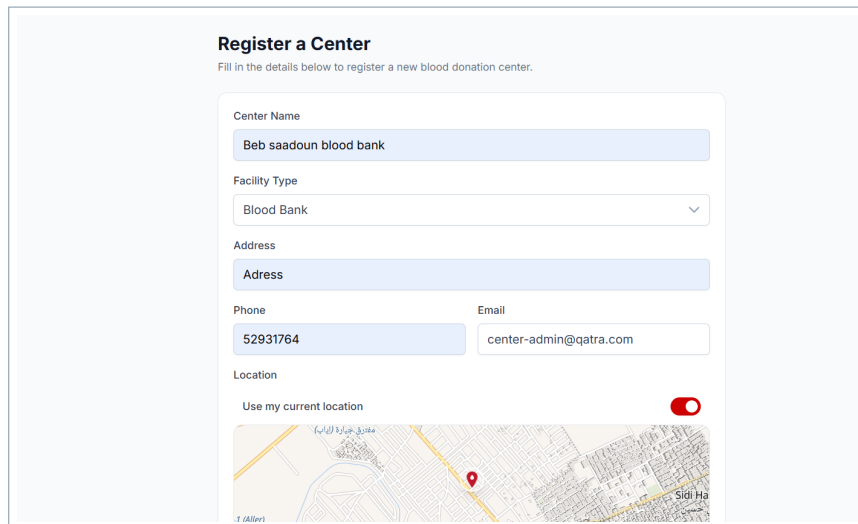


(a) Geographic center discovery



(b) Searchable center list

Figure 5.2: Donation-center discovery interfaces



Register a Center
Fill in the details below to register a new blood donation center.

Center Name
Beb saadoun blood bank

Facility Type
Blood Bank

Address
Address

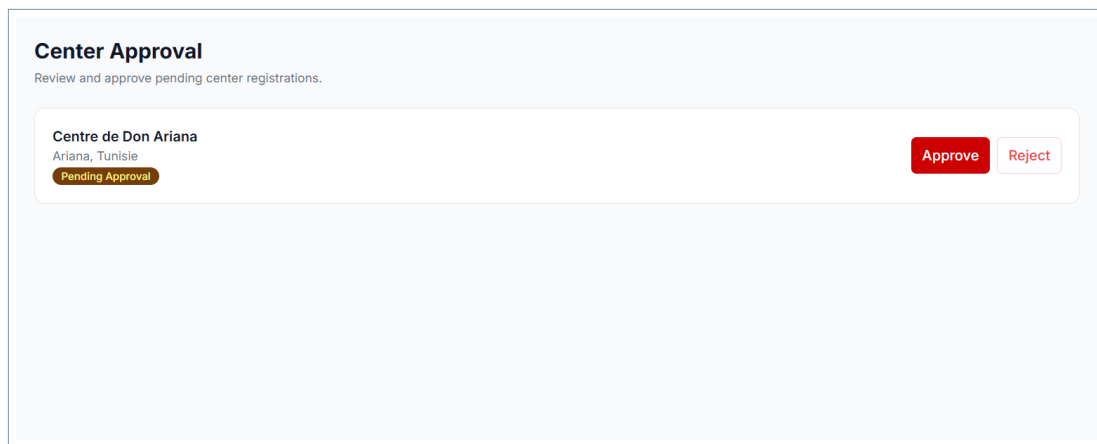
Phone
52931764

Email
center-admin@qatra.com

Location
Use my current location

Map showing location near Sidi Ha

(a) Center registration form



Center Approval
Review and approve pending center registrations.

Centre de Don Ariana
Ariana, Tunisie
Pending Approval

Approve Reject

(b) Administrative approval queue

Figure 5.3: Center registration and approval interfaces

5.1.3 Acceptance Scenarios

- a new user can register and verify the account before protected use;
- invalid credentials or expired reset proof are rejected without revealing sensitive detail;
- public users can consult approved center information;
- a center registration remains pending until authorized approval;
- a center administrator can manage only the relevant center and staff.

5.2 Increment 2: Donor Profiles and Appointments

5.2.1 Objective and Scope

The second increment enabled donors to complete the information required for safe coordination and reserve center capacity. The donor module includes profile, health questionnaire, blood type, location, availability, history, eligibility, certificate, and impact responsibilities. The appointment module includes creation, rescheduling, cancellation, status, health screening, and outcome.

The Angular application provides onboarding pages for health, blood type, location, availability, and notification preferences. It also provides center detail, slot booking, appointment cards, and donor dashboard views. MapLibre supports map display without making location an untyped UI string.

5.2.2 Workflow

The donor selects a center and date, receives available slots, and submits a booking. The backend verifies identity, donor state, center/slot validity, and conflicts. A successful response becomes part of the donor history and center schedule. Rescheduling repeats validation against the target slot.

The screenshot shows the 'Health Screening' step (Step 1) of a three-step onboarding process. The steps are labeled '1 Health', '2 Blood & Location', and '3 Preferences'. The 'Health Screening' section includes two toggle switches: 'Chronic illness?' (set to 'On') and 'On any medication?' (set to 'On'). Below these is a 'Recent history' section with three date input fields: 'Last surgery date', 'Last travel date', and 'Last tattoo/piercing date', each with a placeholder 'jj/mm/aaaa' and a calendar icon. At the bottom, there are 'Skip' and 'Next →' buttons. A footer note states: 'You can complete your profile later from your settings.'

(a) Health-screening questionnaire

The screenshot shows the 'Blood Type' and 'Your Location' steps (Step 2) of the onboarding process. The 'Blood Type' section has a toggle switch 'I don't know my blood type' (set to 'On') and a 'Select your blood type' section with eight buttons: A+, A-, B+, B-, AB+, AB-, O+, and O-. The 'Your Location' section has a toggle switch 'Use my current location' (set to 'On') and a map of Tunisia with labels for various cities like Bab Laassal, Monplaisir, and Carthage. The map is centered on Tunis. The top navigation bar shows '1 Health' as completed, '2 Blood & Location' as the current step, and '3 Preferences' as the next step.

(b) Blood type and location step

Figure 5.4: Donor onboarding workflow

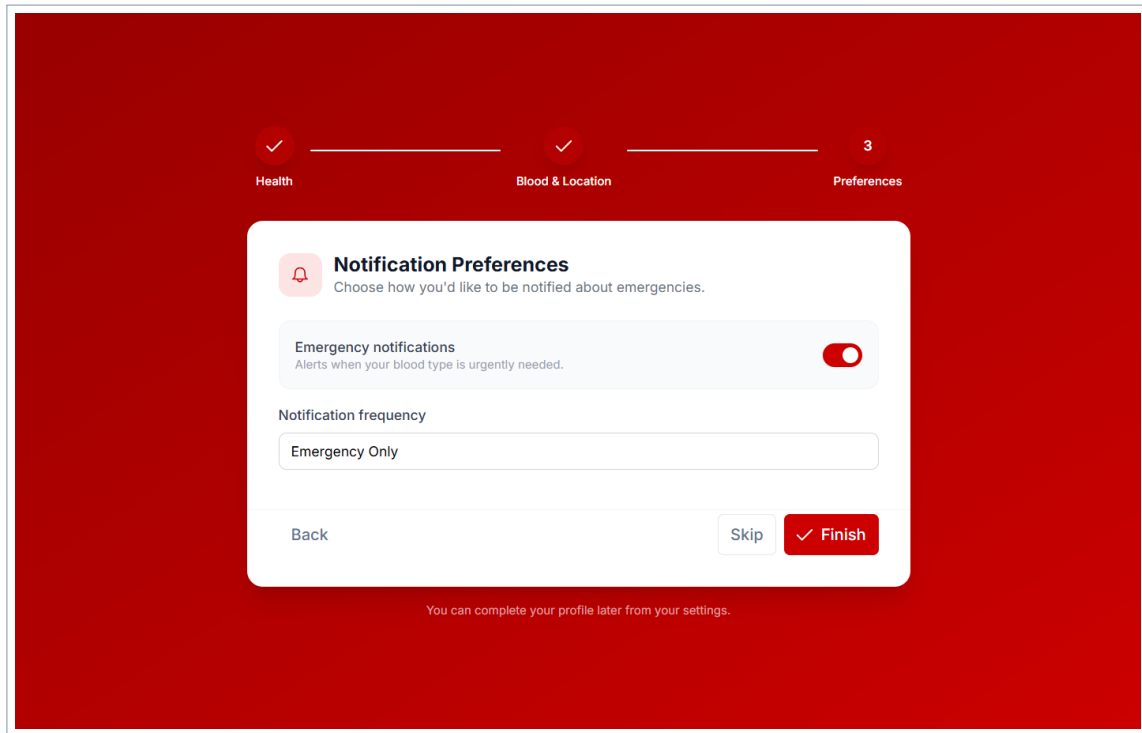


Figure 5.5: Notification-preference configuration

The screenshot shows the 'Book Appointment' form in the Qatra application. The form has a red header with the title 'Book Appointment' and a subtitle 'Choose a center, date, and time slot for your donation.' Below the header, there are three steps: 1. Center, 2. Date & Time, and 3. Confirm. The first step, 'Center', is active. It features a search bar with the placeholder text 'Search by name or city...' and a red 'Search' button. Below the search bar, there is a grid of donation centers. The first center, 'Beb saadoun blood bank' (Tunis, Tunisie), is highlighted with a red border. Other centers include 'Centre de Don Ariana', 'Centre de Don Ben Arous', 'Centre de Don Bizerte', 'Centre de Don Gabes', 'Centre de Don Gafsa', and 'Centre de Don Jendouba'. To the right of the grid, there is a 'Your Selection' box that displays the selected center, date, and time slot. The bottom of the form shows 'Page 1 of 4'.

(a) Donation-center selection

The screenshot shows the 'Book Appointment' form in the Qatra application, now on the 'Date & Time' step. The form has a red header with the title 'Book Appointment' and a subtitle 'Choose a center, date, and time slot for your donation.' Below the header, there are three steps: 1. Center, 2. Date & Time, and 3. Confirm. The second step, 'Date & Time', is active. It features a date picker with a calendar view for September 2026. The date '9' is selected. To the right of the date picker, there is a 'Time Slot' section with four options: '8:00 AM - 8:30 AM' (10 / 10 available), '8:30 AM - 9:00 AM' (10 / 10 available), '9:00 AM - 10:00 AM' (9 / 10 available), and '9:30 AM - 10:00 AM' (10 / 10 available). The '9:00 AM - 10:00 AM' slot is highlighted with a red border. To the right of the date and time selection, there is a 'Your Selection' box that displays the selected center, date, and time slot. The bottom of the form shows a 'Back' button and a 'Continue' button.

(b) Date and available-slot selection

Figure 5.6: Appointment center and slot selection

Qatra Home Appointments History Emergencies Impact Centers

Book Appointment
Choose a center, date, and time slot for your donation.

1 Center — 2 Date & Time — 3 Confirm

Confirm Your Booking

CENTER
Beb saadoun blood bank

DATE
Sep 10, 2026

TIME
8:00 AM – 8:30 AM

TYPE
Regular Donation

Your Selection

Center
Beb saadoun blood bank

Date
Sep 10, 2026

Time
8:00 AM – 8:30 AM

Back Confirm Booking

Figure 5.7: Appointment booking confirmation

5.2.3 Privacy Considerations

Health answers and precise location must not be included in generic list responses. The UI should explain why data is collected and allow notification preferences to be changed. Location improves proximity matching but should not be treated as public profile information.

5.3 Increment 3: Staff Operations and Donation Completion

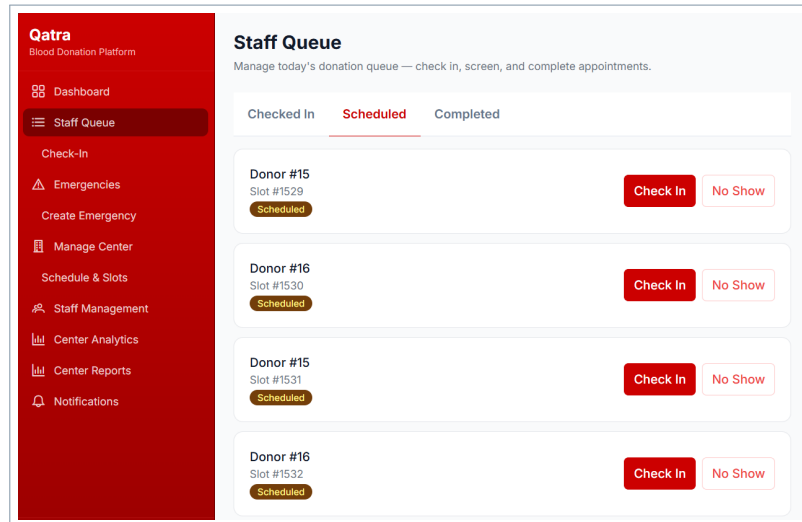
5.3.1 Objective and Scope

This increment connected planned appointments to actual center operations. Staff need a daily schedule and a focused task queue. At arrival, the staff member checks in the donor, consults authorized eligibility information, records screening, and completes or rejects the donation outcome.

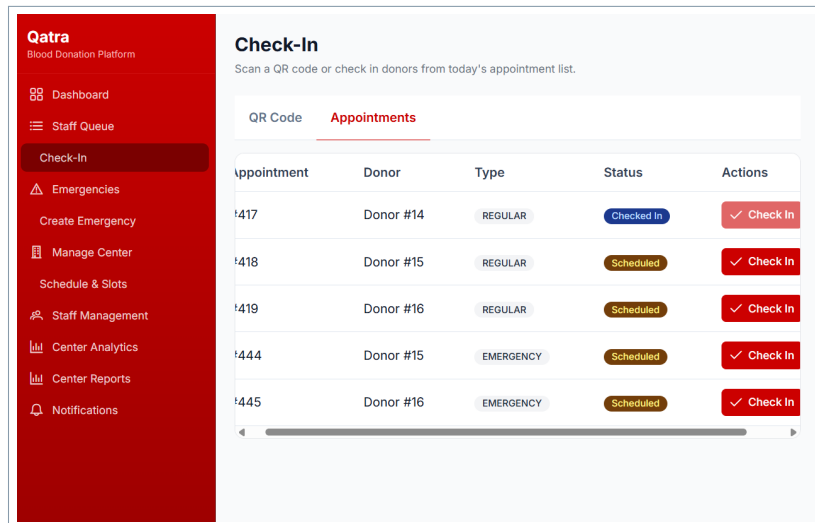
The screening record captures the decision at donation time. It does not replace professional medical judgment. Completing an appointment can update donation history and future eligibility. Marking a donor as no-show can contribute to reliability logic, but automatic penalties should remain transparent and correctable.

5.3.2 Operational Sequence

The staff interface should prioritize today's records, status, scheduled time, and next action. QR-code support can accelerate check-in, but the decoded identifier must still be authorized and validated by the backend. A QR code is a pointer, not proof of eligibility.

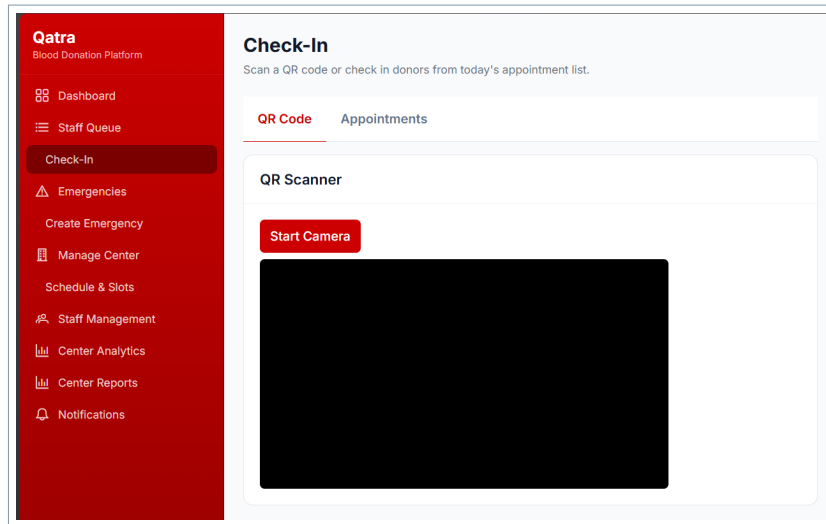


(a) Scheduled donor queue

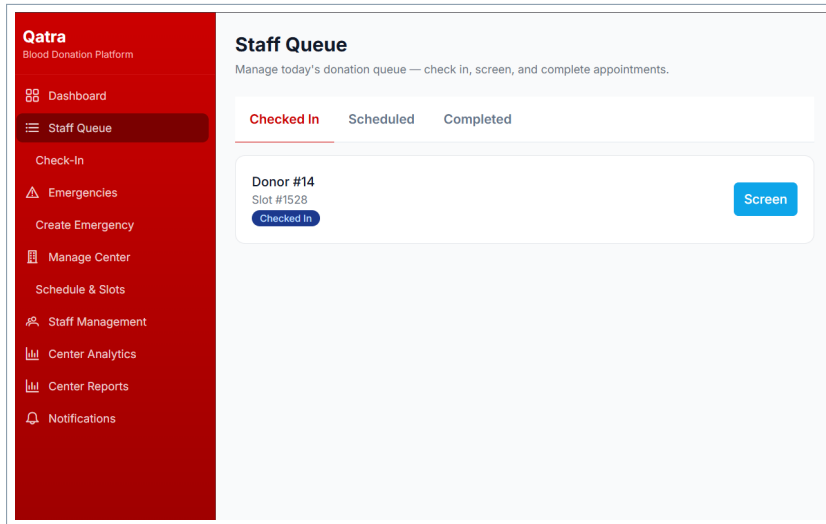


(b) Appointment check-in list

Figure 5.8: Center staff schedule and check-in interfaces

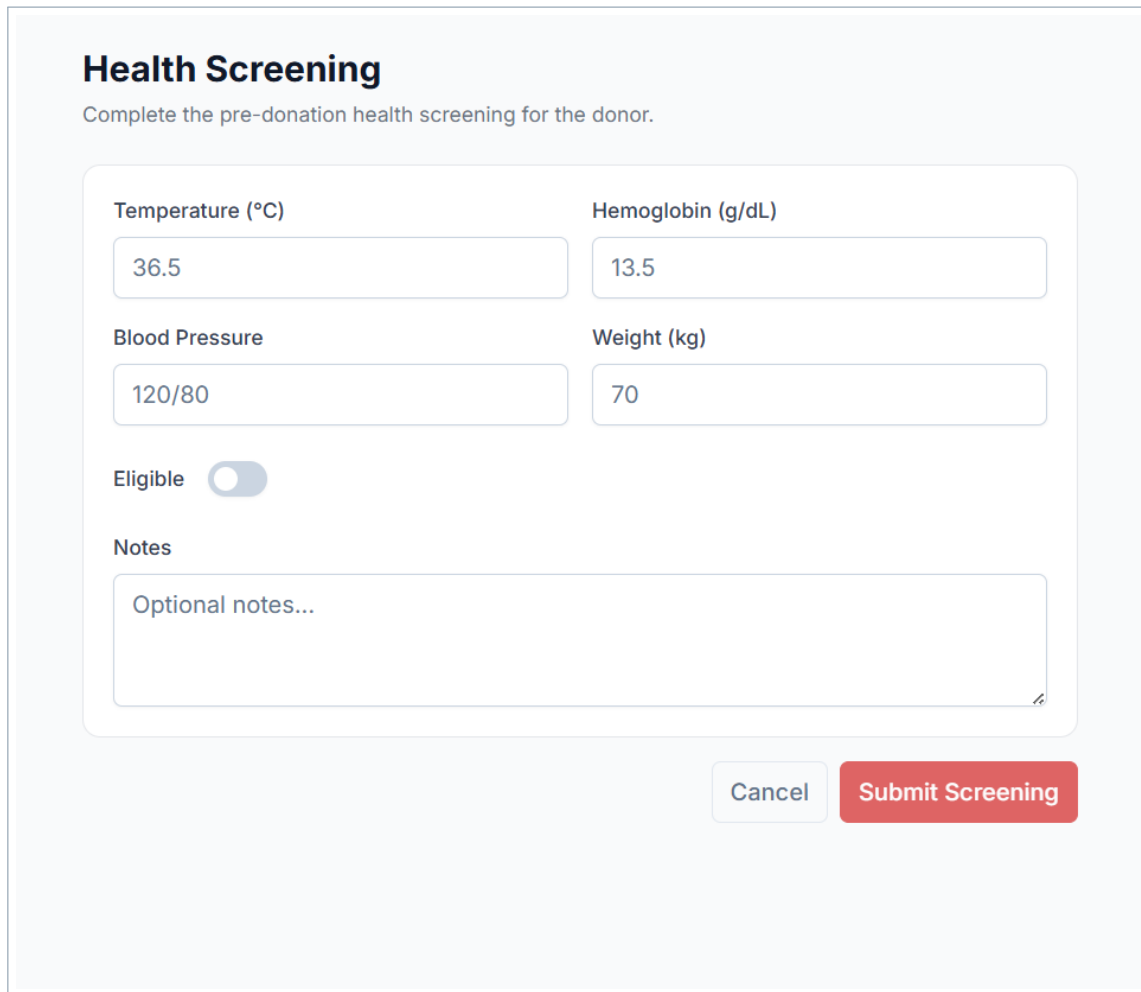


(a) QR-assisted check-in



(b) Checked-in donor ready for screening

Figure 5.9: QR check-in and staff processing queue



Health Screening

Complete the pre-donation health screening for the donor.

Temperature (°C)	Hemoglobin (g/dL)
36.5	13.5
Blood Pressure	Weight (kg)
120/80	70
Eligible ☐	
Notes	
Optional notes...	

Figure 5.10: Health screening form

5.3.3 Auditability

Screening, completion, no-show, and staff administration are state-changing actions. Audit records should capture the authenticated actor, action, target, timestamp, and safe metadata. Sensitive questionnaire content should not be copied wholesale into generic audit text.

5.4 Increment 4: Emergency Management

5.4.1 Objective and Scope

An emergency record expresses a need that cannot wait for normal appointment discovery. It includes required blood information, location, deadline, and lifecycle status. Authorized users create the request, follow its progress, and resolve or expire it. Donors receive only relevant information required to make an informed response.

The emergency lifecycle must prevent invalid transitions. A resolved or expired request should not continue dispatching new alerts. Acceptance by a donor is not equivalent to a completed

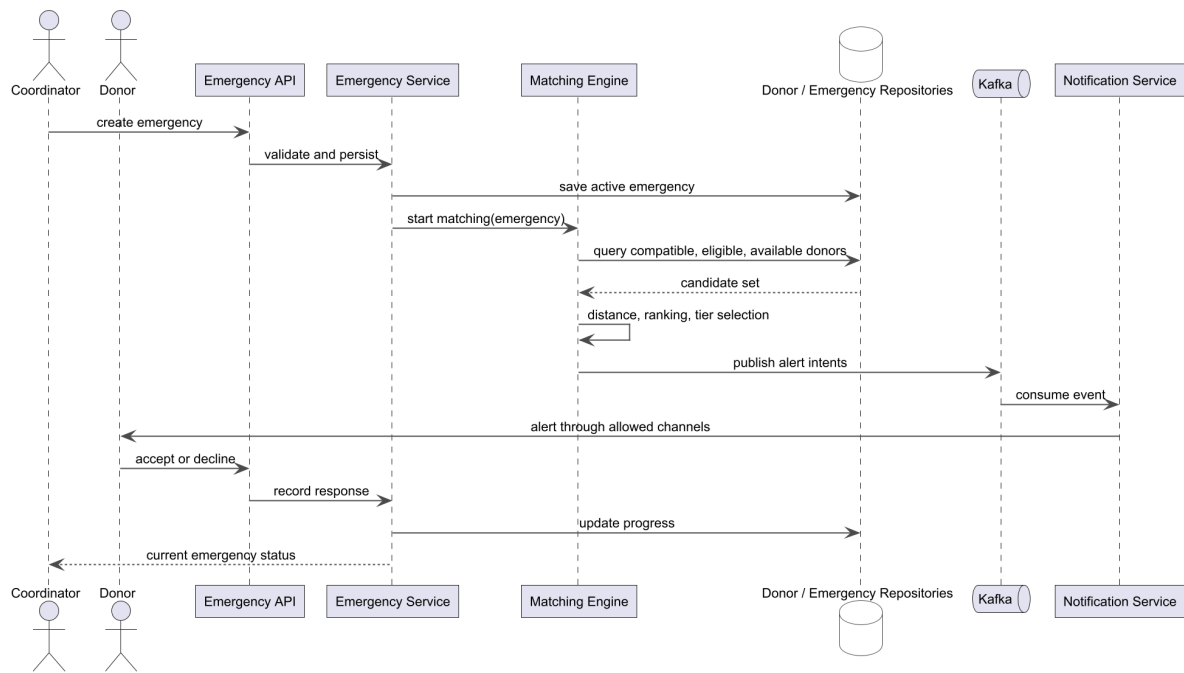


Figure 5.11: Emergency creation, candidate matching, and response sequence

donation; it indicates intent and supports coordination. Clinical compatibility and screening remain center responsibilities.

Emergencies
17 emergency requests for your center.

Status: All

ID	Blood Type	Units	Urgency	Status	Created At
#18	O_NEGATIVE	6	CRITICAL	Open	Sep 13, 2026, 12:00
#17	B_POSITIVE	3	HIGH	Open	Sep 12, 2026, 12:00
#16	A_POSITIVE	4	MEDIUM	Open	Sep 11, 2026, 11:00
#15	AB_NEGATIVE	2	HIGH	Open	Sep 10, 2026, 12:00
#14	O_POSITIVE	5	CRITICAL	Open	Sep 9, 2026, 1:00
#13	B_NEGATIVE	4	MEDIUM	Open	Sep 9, 2026, 12:00
#12	A_NEGATIVE	3	HIGH	Expired	Sep 8, 2026, 1:00
#11	AB_POSITIVE	6	CRITICAL	Expired	Sep 8, 2026, 12:00
#9	A_POSITIVE	5	MEDIUM	Expired	Sep 8, 2026, 11:00

(a) Emergency request list

Create Emergency
Submit a new blood donation emergency request for a center.

Center
Beb saadoun blood bank

Blood Type: B+ Units Needed: 7

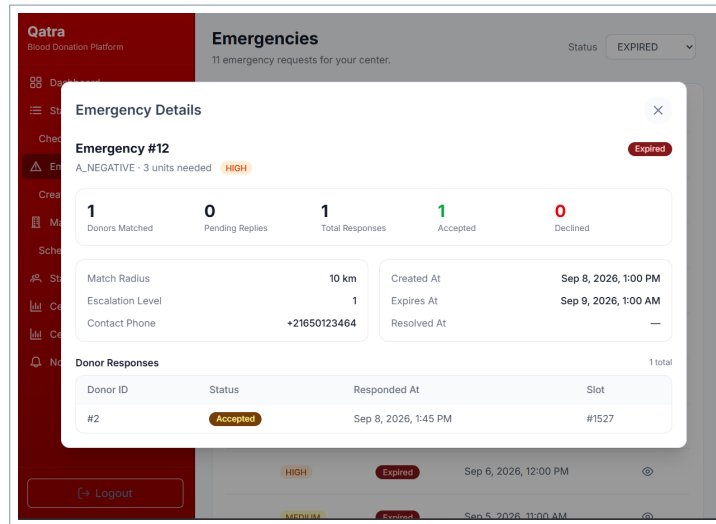
Urgency
☐ Critical ☐ High ☒ Medium ☐ Low

Contact Phone: 24565478 Match Radius (km): 50

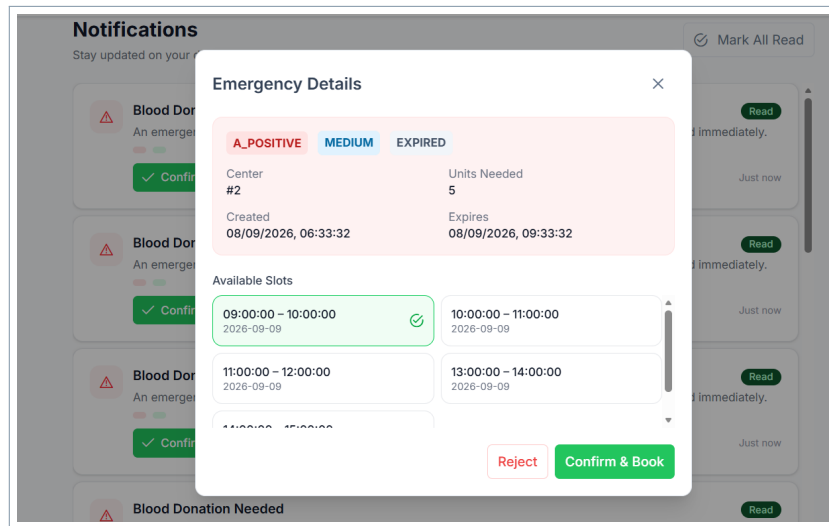
Cancel Create Emergency

(b) Emergency creation form

Figure 5.12: Emergency management and request creation



(a) Emergency progress and response summary



(b) Donor response and slot selection

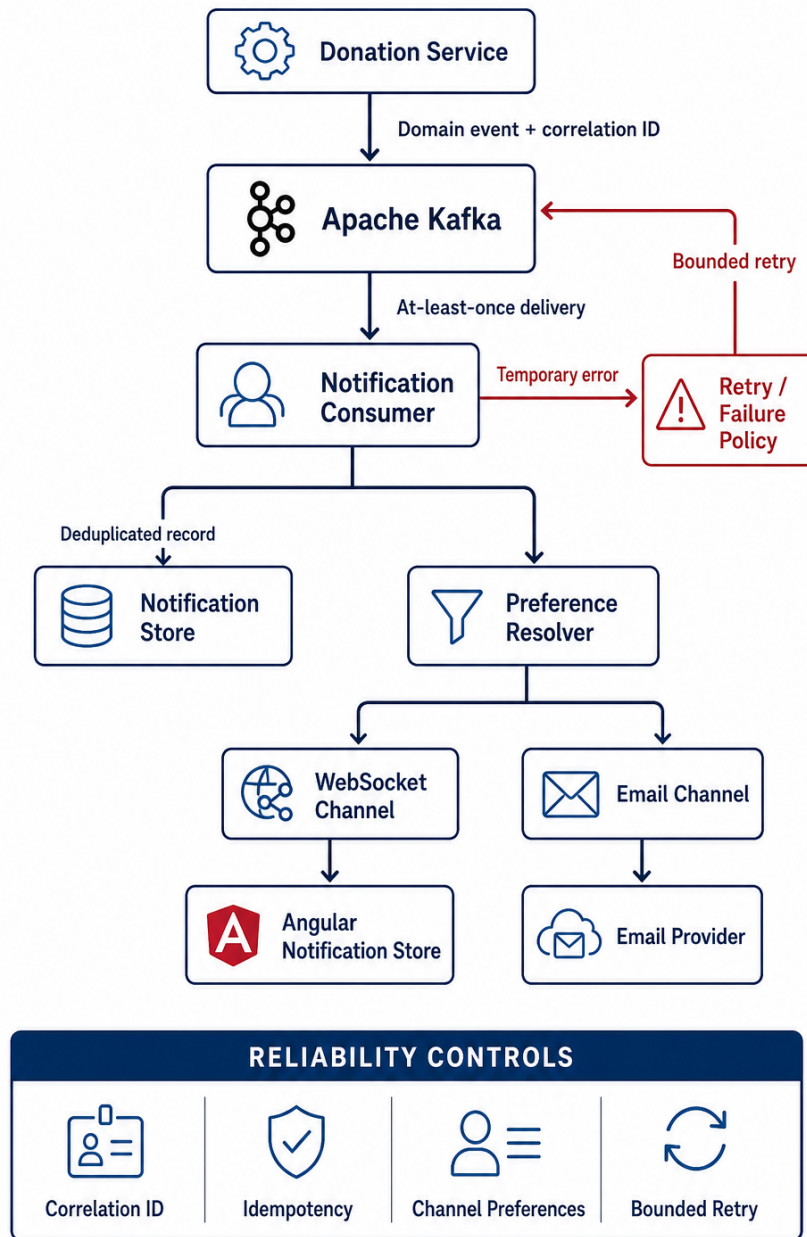
Figure 5.13: Emergency progress and donor-response workflow

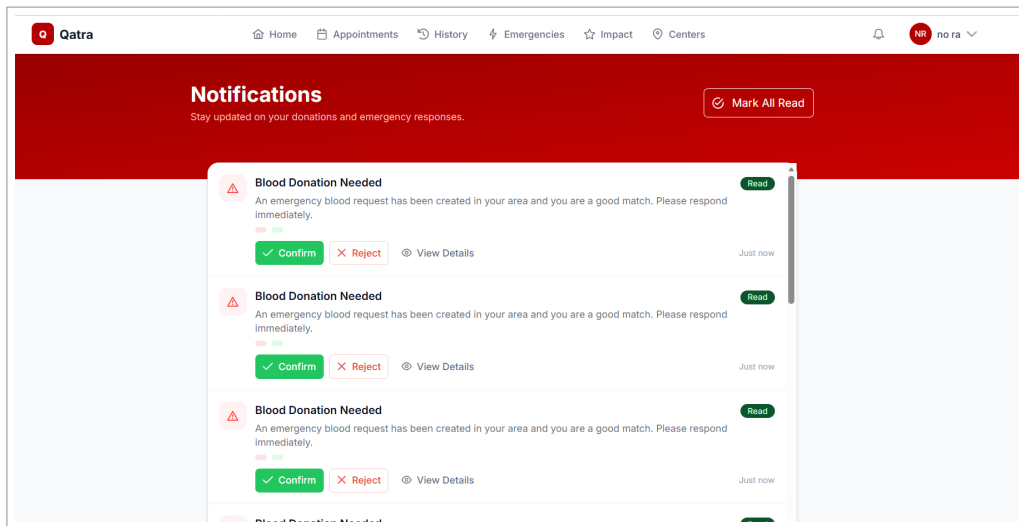
5.5 Increment 5: Matching and Notifications

5.5.1 Objective and Scope

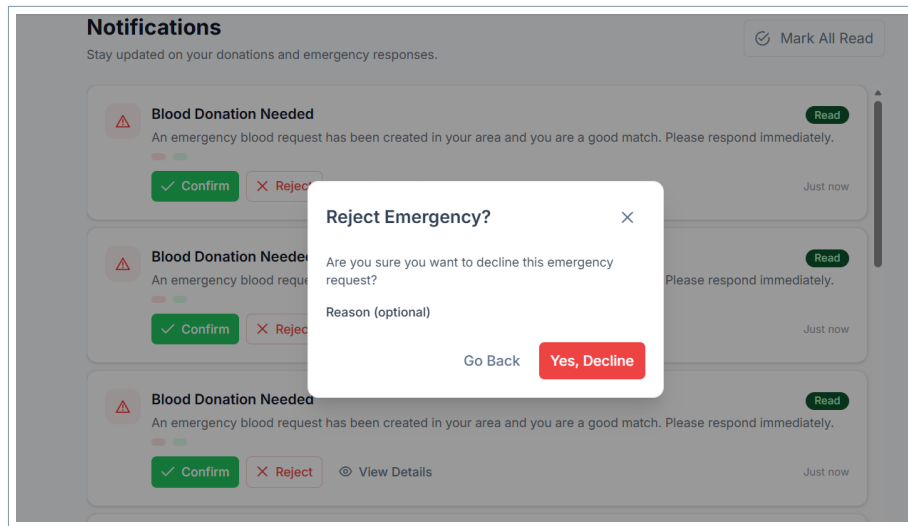
The fifth increment transformed an emergency into targeted communication. Matching considers blood compatibility, availability, eligibility, geographical distance, and reliability-related information. Candidates are ranked and processed in tiers. If the response is insufficient, the search radius can expand until a configured limit or deadline is reached.

Notification work is transferred through Kafka so that API response time is not coupled to delivery channels. The notification service can persist delivery state, apply preferences, and deliver real-time updates over WebSocket. Failure is observable and retryable without recreating the emergency.

KAFKA EVENT AND NOTIFICATION DELIVERY FLOW**Figure 5.14:** Kafka event and notification delivery flow

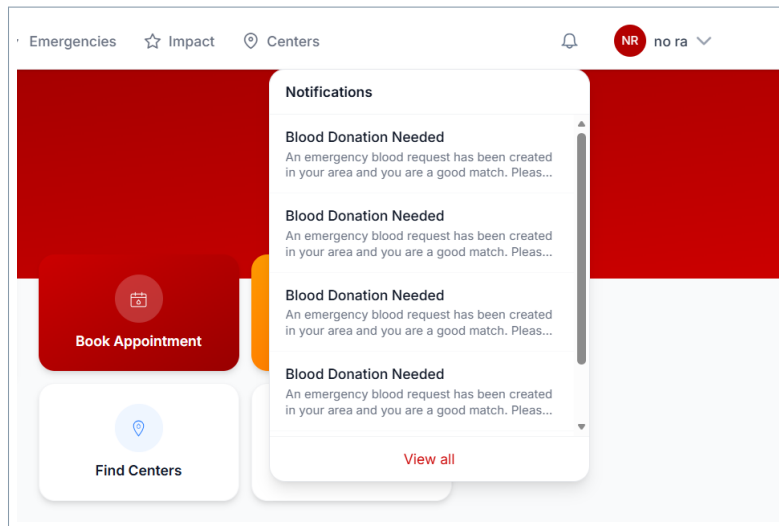


(a) Emergency alerts in the notification center

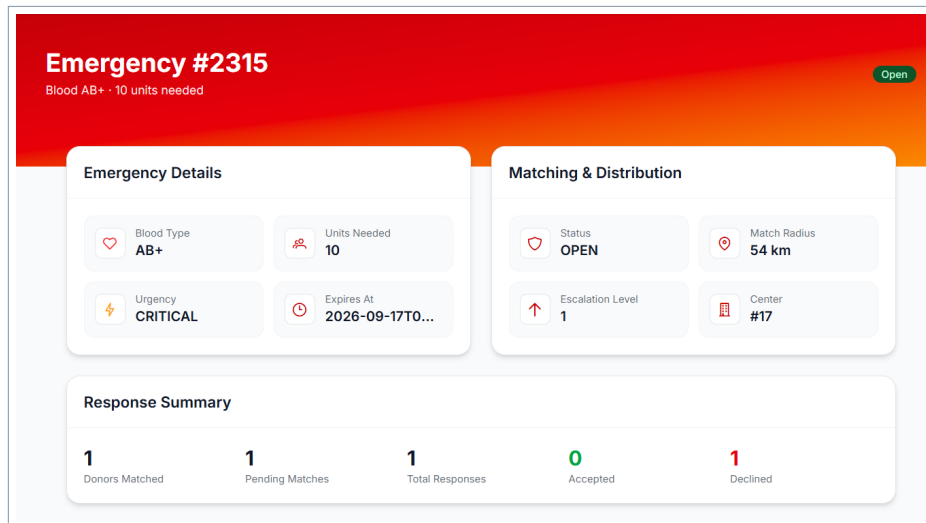


(b) Controlled emergency-decline dialog

Figure 5.15: Donor emergency alert and response interfaces



(a) Header notification popup



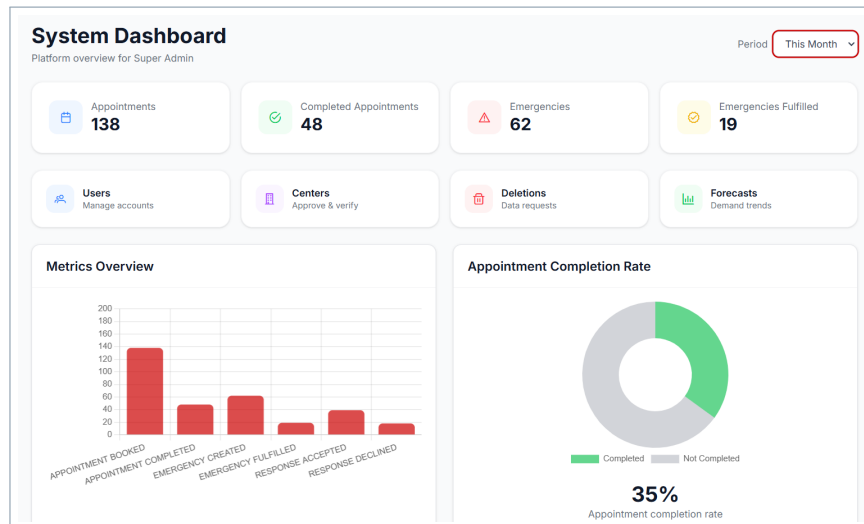
(b) Donor emergency details

Figure 5.16: Notification access and emergency-status interfaces

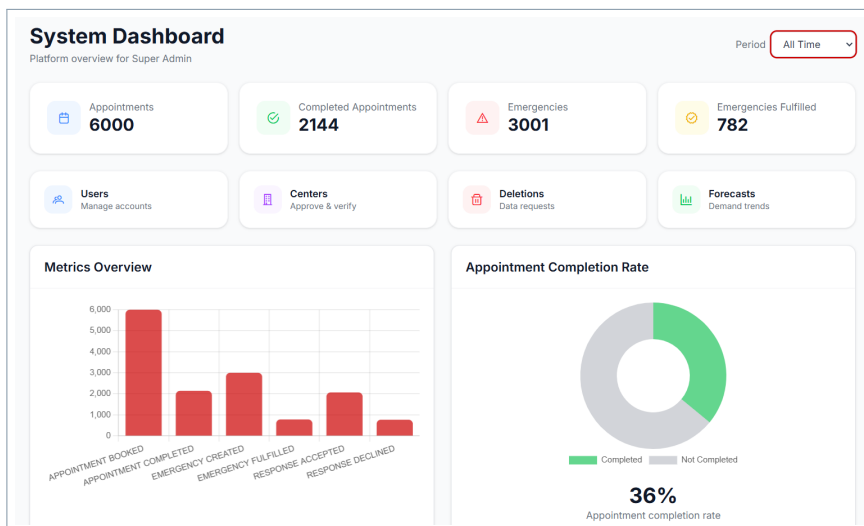
5.6 Increment 6: Analytics, Administration, and Operations

The final grouping completed cross-cutting administrative and operational capabilities. Center administrators consult activity and analytics, manage staff and slots, and generate reports. Super administrators govern users, roles, center approvals, health restrictions, deletion requests, system dashboards, and audit logs.

Analytics must distinguish operational summaries from clinical prediction. Counts of appointments, completed donations, no-shows, active emergencies, and delivery states can be calculated from platform records. Claims about regional blood demand or medical impact require validated data and are treated as future analytical work when evidence is absent.



(a) Current-month operational indicators



(b) All-time operational indicators

Figure 5.17: Super-administration analytics dashboards

Qatra
Blood Donation Platform

- Admin Dashboard
- Users
- Center Approval
- Health Restrictions
- Audit Logs**
- System Health
- Reports
- Config
- Notifications

Audit Logs
Track all system actions and changes for accountability.

Action: **All Actions** From: To: **Filter** Clear

Timestamp	Action	Entity	User
15 Sept 2026, 09:30:00	APPOINTMENT_BOOKED	Appointment	#70
14 Sept 2026, 11:30:00	APPOINTMENT_BOOKED	Appointment	#69
14 Sept 2026, 10:30:00	APPOINTMENT_BOOKED	Appointment	#68
14 Sept 2026, 09:30:00	APPOINTMENT_BOOKED	Appointment	#67
13 Sept 2026, 13:30:00	APPOINTMENT_BOOKED	Appointment	#54
13 Sept 2026, 12:45:00	DONOR_RESPONSE	DonorResponse	#68
13 Sept 2026, 12:00:00	EMERGENCY_CREATED	EmergencyRequest	#55
13 Sept 2026, 11:30:00	APPOINTMENT_BOOKED	Appointment	#66

Figure 5.18: Super-administration audit log interface

The same increment includes deployable containers, k3s manifests, CI workflows, and observability configuration. These do not appear directly in a donor workflow, but they determine whether the service can be maintained and diagnosed.

5.7 Increment Review

Increment	User value	Principal dependency	Residual risk
1	Trusted users and managed centers	Security and center domain	Verification and authorization edge cases
2	Donor readiness and planned visits	Identity, centers, slots	Concurrent booking and health privacy
3	Executed donation workflow	Appointments and staff roles	Screening correctness and audit scope
4	Trackable urgent request	Donor and center data	Invalid transitions and deadline handling
5	Targeted, decoupled alerts	Emergency events and preferences	Duplicate delivery and retry exhaustion
6	Governance and operability	All domain records and deployment	Evidence depth and production hardening

Table 5.1: Kanban increment review and residual risks

Conclusion

The increments trace a dependency-aware path from identity to emergency coordination and operations. Chapter 6 examines donor matching and reliable notification in depth.

CHAPTER 6

EMERGENCY MATCHING AND NOTIFICATION INTELLIGENCE

Introduction

Emergency coordination is Qatra’s most time-sensitive capability. A broad notification blast is easy to implement but produces fatigue, irrelevant alerts, and weak accountability. Qatra instead models matching as a staged decision process and notification as a separate delivery responsibility.

6.1 Matching Inputs

The candidate set is constrained by information from several domains:

- blood-type compatibility with the emergency requirement;
- active account and donor profile state;
- completed onboarding and known eligibility information;
- current availability and notification preferences;
- next eligible date and permanent restriction state;
- donor and emergency locations;
- prior response or reliability information where implemented;
- emergency deadline and current lifecycle state.

Filtering should remove candidates who cannot safely or legitimately be contacted. Ranking should order remaining candidates; it must not reintroduce filtered candidates.

6.2 Blood Compatibility

Compatibility is not equivalent to identical blood type. Red-cell donation rules allow some donor types to serve multiple recipient types, but the exact operational rule depends on the requested product and clinical policy [23]. The World Health Organization also recommends mandatory screening of donated blood for transfusion-transmissible infections and testing for ABO and RhD groups [22]. The platform model uses blood-type relationships to reduce irrelevant outreach; final clinical decisions remain under qualified staff responsibility.

Compatibility logic should be centralized and tested as a domain function. Duplicating it in the frontend, SQL, and notification service would create inconsistent behavior. The frontend may display explanatory information, but the backend remains authoritative.

6.3 Eligibility and Availability

Eligibility combines permanent and temporary conditions. A completed donation can set a future eligible date. A health restriction can prevent matching until reviewed. Availability is a donor preference and may change more frequently than medical status. The matcher therefore evaluates both categories at processing time rather than trusting an old candidate export. These software checks support, but never replace, professional donor-suitability assessment [21].

A scheduled job may restore eligibility when a cooldown passes. Such a job must be idempotent: running it twice should not produce duplicate history or notifications. Changes to permanent restrictions should be auditable and limited to authorized roles.

6.4 Geographical Distance

Qatra compares the donor's location with the location of the emergency to estimate how close the donor is. This estimate helps the platform contact nearby compatible donors first. It represents direct geographical distance; it does not predict the actual travel time because roads, traffic, and available transport are not considered. For privacy, precise donor coordinates are not exposed in emergency notifications.

6.5 Ranking and Tiered Expansion

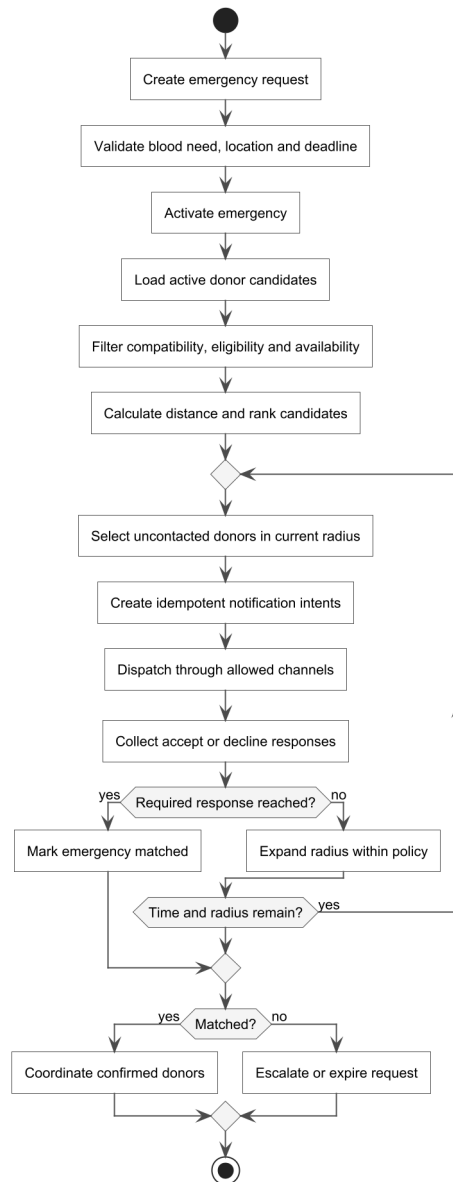
After compatibility and eligibility checks, Qatra gives priority to donors who are nearby, available, reliable, and willing to receive emergency requests through the selected communication channel. These criteria make the matching decision understandable without relying on a complex or opaque formula.

The search starts within the nearest configured area. If too few donors accept the request, Qatra gradually expands the search to a wider area. Donors who have already been contacted are recorded so that they do not receive the same alert repeatedly.

Stage	Candidate scope	Action	Stop condition
Initial	Closest eligible donors	Rank and notify first group	Required responses or wait threshold
Expanded	Wider radius, excluding contacted donors	Notify next ranked group	Required responses, max radius, or deadline
Final	Remaining allowed radius	Last controlled outreach	Resolution, expiry, or operator action

Table 6.1: Conceptual tiered emergency matching strategy

Figure 6.1 expresses the same policy as an activity flow with explicit responsibility partitions, decisions, radius expansion, and termination conditions.

**Figure 6.1:** Emergency matching and notification activity diagram

6.6 Event Publication and Consistency

After candidate selection, a domain action creates notification intent. Publishing directly after a database commit can lose an event if the broker is unavailable; publishing before commit can announce work that later rolls back. A transactional outbox is a common future hardening pattern: persist domain state and an outbox record atomically, then publish asynchronously [17].

The available project uses Kafka-oriented communication, but the report does not claim a complete outbox implementation unless present in the source. This distinction is important in an academic assessment: an architectural recommendation is not presented as completed functionality.

6.7 Notification Channels

The notification service can support in-application updates, WebSocket delivery, and email-oriented communication. Channel selection respects donor preferences and the urgency policy. An in-app record provides durable history, while WebSocket provides immediate update when the user is connected. Email can reach an offline user but depends on an external provider and deliverability.

Notification payloads should contain the minimum necessary data. An emergency alert may include blood need, approximate area, deadline, and response action without exposing another person's medical identity.

6.8 Delivery State, Retry, and Idempotency

Delivery has more states than sent or failed. A robust model distinguishes created, queued, dispatched, acknowledged by provider, delivered where supported, read, responded, retryable failure, and terminal failure. The implemented model may use a subset, but the distinction guides observability.

At-least-once broker delivery means consumers must tolerate duplicates. An event identifier or compound business key can prevent duplicate notification records. Retry should use bounded exponential backoff and classify permanent errors such as invalid addresses separately from temporary provider or network failures. Exhausted retries belong in a reviewable dead-letter path.

6.9 Real-Time Updates

WebSocket connections allow notification counts, emergency progress, and status changes to appear without polling. The authenticated connection must be associated with the correct user, and subscription destinations must not permit access to another donor's messages. Reconnection should refresh authoritative state because messages can be missed while disconnected.

6.10 Reliability Score and Fairness

The use-case model includes a reliability score between 0 and 100. Such a score can help order candidates, but it must not punish users for circumstances beyond their control or become an unexplained barrier. Inputs should be limited, interpretable, and reviewable. Declining an emergency should not automatically be treated as misconduct; explicit availability and response history need contextual treatment.

Fairness also concerns geography. Always selecting the closest highly reliable donors can repeatedly burden the same group. Rotation, recent-contact limits, and preference-aware ranking can reduce fatigue. These are governance choices that should be documented and evaluated with real operational data before production use.

6.11 GDPR and Audit

Matching processes location, health-derived eligibility, and behavior. Under the GDPR, health data belongs to a special category and personal-data processing must respect principles including purpose limitation and data minimization [24]. Donors should be able to update location and preferences and request account deletion. Administrators need a controlled workflow to reconcile deletion with legally required records. Audit logs should record decisions without becoming a second uncontrolled health-data store.

6.12 Failure Scenarios

Failure	Risk	Response
Kafka unavailable	Notification event cannot be dispatched	Retry publication; use health monitoring; consider outbox
Notification provider timeout	Unknown or delayed delivery	Bounded retry with idempotency key
WebSocket disconnected	User misses immediate update	Persist notification and refresh on reconnect
Stale donor availability	Irrelevant alert	Re-evaluate current state before dispatch
Emergency resolved during expansion	Late notifications	Check emergency state before every tier/batch
Duplicate event	Repeated alert and fatigue	Deduplicate by event/business identifier

Missing coordinates	Distance cannot be calculated	Exclude from proximity tier or use explicit fallback policy
Clock or deadline mismatch	Alerts after expiry	Store UTC timestamps and validate centrally

Table 6.2: Emergency matching and notification failure analysis

Conclusion

Targeted matching combines domain constraints, geographical reasoning, transparent ranking, and staged expansion. Reliable notification adds asynchronous delivery, idempotency, retry, privacy, and real-time authorization. The final chapter examines whether the surrounding engineering system can build, deploy, observe, and validate these capabilities responsibly.

CHAPTER 7

INDUSTRIALIZATION, TESTING, AND OPERATIONAL READINESS

Introduction

A distributed application is incomplete if it can only be explained from source code. Repeatable builds, controlled deployment, metrics, logs, traces, and explicit verification limits are part of the engineering result. This chapter evaluates the artifacts available in the three Qatra repositories.

7.1 Repository and Delivery Organization

The project is divided into three extracted repository folders:

- `qatra-master`: donation service, notification service, PostgreSQL resources, Compose orchestration, CI and deployment workflows;
- `qatra_angular-master`: Angular frontend, environment-generation script, Docker and Nginx resources, frontend workflow;
- `qatra_infra-master`: k3s manifests using Traefik routing, plus Prometheus, Grafana, Loki, Promtail, Jaeger, and administration resources.

This separation supports different delivery concerns, but release compatibility must be managed. API changes, image tags, environment variables, and manifest versions should be coordinated rather than assuming every latest image is compatible.

7.2 Continuous Integration

The backend CI workflow runs for pull requests to the configured branches. A matrix creates one job for each service, installs Temurin Java 21 with Maven caching, and executes the Maven

wrapper tests. This provides isolation: a failure identifies the affected service while keeping the workflow definition concise.

The local Windows inspection found a Maven wrapper startup error before tests could execute. The error occurred in the generated wrapper's PowerShell logic while accessing a filesystem target property. This report therefore records test inventory and CI intent but does not present local passing results.

7.3 Continuous Deployment

The deployment workflow responds to changes on the master branch. A path filter determines which service changed. The corresponding job authenticates to GHCR, builds and pushes a Docker image, connects to the server by SSH, imports the image into k3s, tags it for the local runtime, and restarts the deployment.

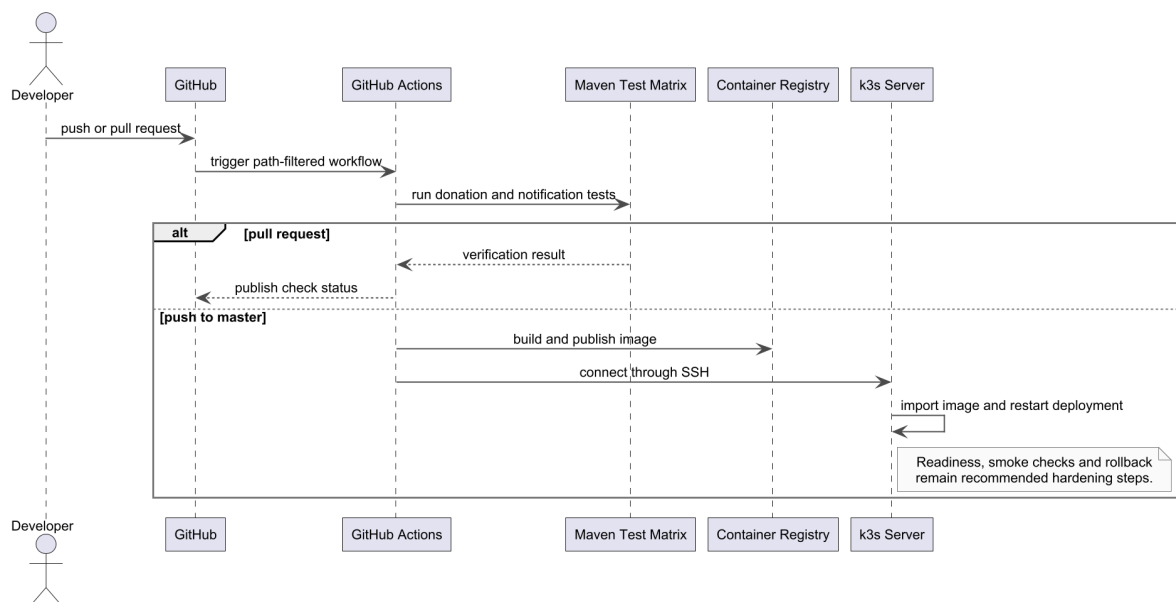


Figure 7.1: Qatra CI/CD pipeline from change to k3s rollout

The workflow demonstrates automated deployment intent. Production hardening should add immutable version tags, rollout status verification, post-deployment health tests, and rollback to a known image. A restart alone does not prove that the new pods became ready or that business workflows remain valid.

7.4 Container and Configuration Management

The root Compose configuration defines the local distributed environment. Validation of the Compose model succeeded when syntax was checked, but numerous required environment variables were unset without an environment file. These include database credentials, JWT secrets, service URLs, Grafana credentials, and notification provider settings. This is expected for a template-driven repository, but setup documentation must make it explicit.

Secrets must not be committed in Compose or Kubernetes manifests. Example environment files should contain names and safe placeholders only. Production secrets should be supplied through a controlled mechanism and rotated when exposure is suspected.

7.5 k3s Deployment

The infrastructure manifests place the two application services and their operational dependencies in the single qatra namespace. Traefik IngressRoute resources expose the APIs and WebSocket endpoint; Nginx is used only by the local Compose topology. Deployment order matters: namespace and stateful dependencies precede application workloads; metrics/logging backends precede dashboards; ingress follows services.

The provided `deploy.sh` represents an operator entry point. A robust procedure should verify context and namespace, apply manifests declaratively, wait for readiness, and display failing pods or events. Administrative tools such as pgAdmin, RedisInsight, and Kubernetes Dashboard require strong access controls and should not be exposed publicly by default.

7.6 Observability Architecture

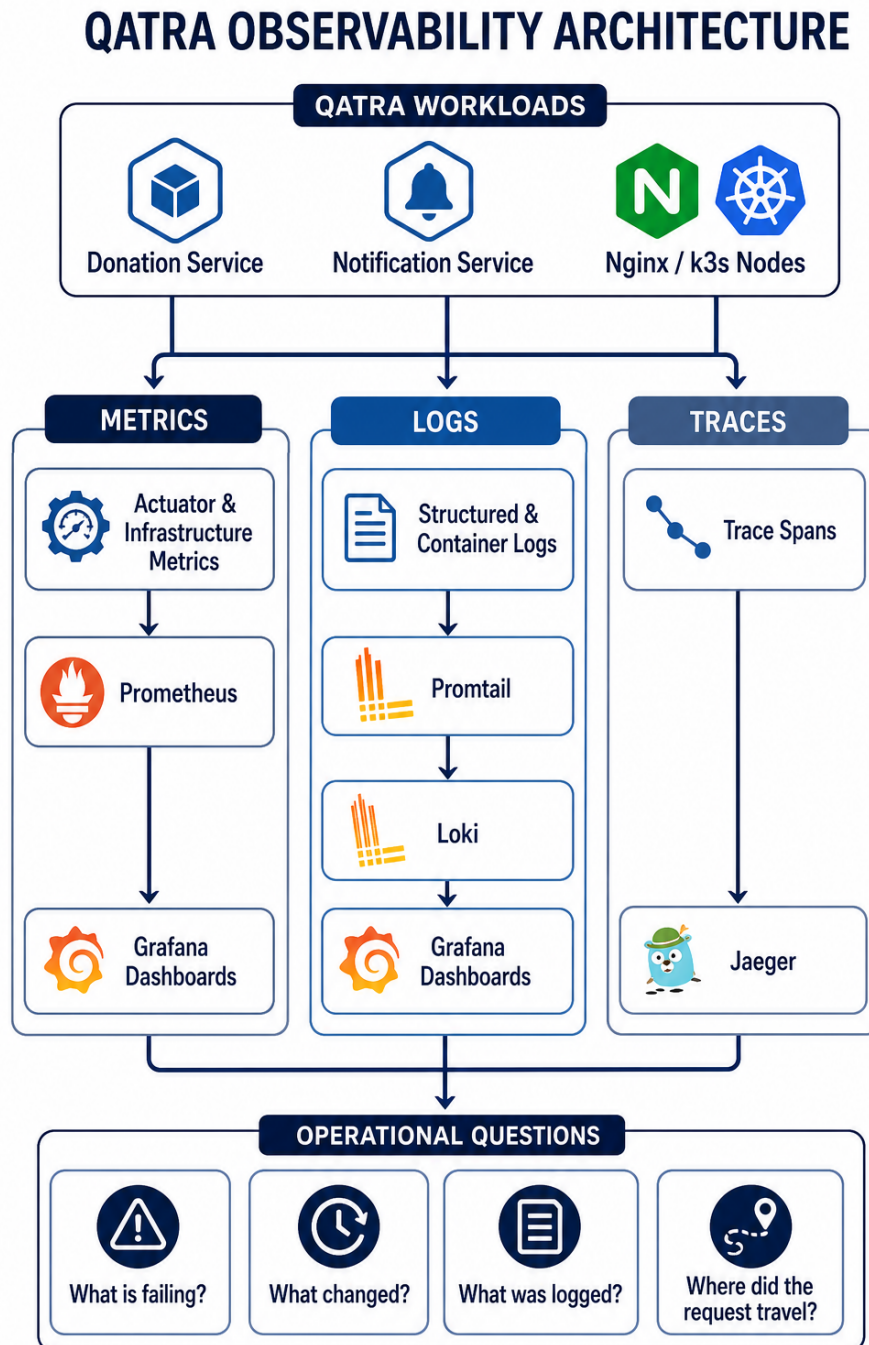


Figure 7.2: Metrics, logs, and traces in the Qatra environment

Prometheus collects time-series data and makes it available to Grafana dashboards. Loki stores logs shipped by Promtail. Jaeger visualizes traces across service boundaries. These signals should share service, environment, instance, and correlation labels. High-cardinality values such as user IDs or raw URLs should not become metric labels.

Useful service-level indicators include request rate, error rate, latency, JVM memory,

thread pools, database connection pools, Kafka consumer lag, notification retry count, WebSocket connections, and emergency matching duration. The repository contains monitoring configuration and dashboards; this report does not invent measured values from an unavailable running environment.

7.7 Automated Test Inventory

Static inspection identified 31 Java test files in the donation service, six Java test files in the notification service, and one Angular specification file. The count refers to files, not necessarily individual test methods. It demonstrates significant backend-oriented test structure and a clear frontend coverage gap.

Component	Test files	Interpretation
Donation service	31	Domain/application/web checks are present; execution was blocked locally by wrapper startup
Notification service	6	Service-focused test structure exists; execution was blocked locally by the same wrapper issue
Angular frontend	1	Test infrastructure exists, but feature coverage is insufficient for the application size

Table 7.1: Observed automated test-file inventory

7.7.1 Reproducibility Verification Performed for This Report

The commands in Table 7.2 were executed in the supplied Windows workspace in August 2026. A failed prerequisite or wrapper startup is reported as such and is not counted as a failed application test.

Check	Observed evidence	Conclusion
Java runtime	OpenJDK 21.0.11 was detected	Required backend runtime is available
Backend tests	<code>mvnw.cmd -B test</code> stopped in the wrapper's PowerShell bootstrap with a null-array error	Maven never started; no backend test pass or failure is claimed
Frontend production build	Angular reported a missing application-builder package because dependencies were not installed	Compilation did not start; no successful frontend build is claimed
CI definition	GitHub Actions matrix targets donation-service and notification-service with Java 21 and Maven tests	Workflow intent is verified; no exported run result was available
k3s deployment	Manifests define one qatra namespace, Traefik routes, two application services, and operational dependencies	Static deployment topology is verified; no live rollout result is claimed

Table 7.2: Reproducible checks and their observed outcomes

7.8 Recommended Test Strategy

7.8.1 Domain and Unit Tests

Pure domain tests should cover compatibility, eligibility dates, appointment transitions, emergency transitions, tier expansion, ranking, and reliability bounds. These tests are fast and should avoid Spring context startup.

7.8.2 Persistence and Integration Tests

Repository adapters should be tested against PostgreSQL-compatible behavior, including constraints, pagination, and concurrent slot updates. Kafka integration tests should verify serialization, consumer behavior, duplicate handling, and retry classification. Testcontainers is a suitable future approach for reproducible dependencies.

7.8.3 API and Security Tests

Controller tests should verify validation, error contracts, role enforcement, ownership, and filtering of sensitive fields. Negative cases are essential: a valid token with the wrong center scope must be rejected.

7.8.4 Frontend Tests

The Angular application needs unit tests for stores, services, schemas, guards, and critical components. End-to-end tests should cover registration, login, onboarding, appointment booking, staff completion, emergency creation, donor response, and administrative approval. Browser tests must use isolated data and should not depend on real email providers.

7.8.5 Operational Tests

Deployment verification should include Compose configuration, Kubernetes schema checks, image scanning, startup health, database connectivity, Kafka connectivity, and a minimal API smoke test. Observability tests can verify that actuator metrics are scraped and that a correlated request produces searchable logs and a trace.

7.9 Security and Production Readiness

Area	Available foundation	Required hardening
Authentication	Spring Security and JWT dependencies	Key rotation, revocation strategy, brute-force controls guided by OWASP API Security [25]
Authorization	Roles and protected application features	Systematic object- and resource-scope tests [25]
Secrets	Environment-driven configuration	Managed secrets, rotation, and leak scanning
Containers	Service Dockerfiles and registry workflow	Non-root images, SBOM, vulnerability scanning, immutable tags
Kubernetes	Namespaced manifests and ingress	Network policies, resource limits, pod security, backup tests
Data	PostgreSQL persistence and GDPR workflows	Encryption, minimization, retention policy, restore drills, legal review [24]

Events	Kafka infrastructure	Schema governance, idempotency, dead-letter operation
Observability	Metrics, logs, traces, dashboards	Alerts, retention, PII redaction, on-call procedures

Table 7.3: Production-readiness foundation and hardening roadmap

7.10 Problems Encountered and Corrective Directions

7.10.1 Build Reproducibility

The Maven wrappers use script-only distribution bootstrap. In the inspected Windows environment, the generated PowerShell logic fails before Maven starts. Corrective options are to update the wrapper, add the wrapper JAR distribution mode, or execute in the Linux CI environment. The correct fix should be verified in both services.

7.10.2 Frontend Dependency Setup

The frontend includes a lock file but no installed `node_modules` directory, which is normal for a clean source archive. A reproducible setup should use the lock file and a supported Node version. The build script generates development and production environment files before Angular compilation.

7.10.3 Environment Completeness

Compose configuration is structurally valid but depends on variables. A setup checklist should copy the example file, supply secure values, verify ports, and confirm external notification credentials. Validation must ensure placeholders are not accepted in production.

7.10.4 Evidence Gaps

The source archive contains no application screenshots and no exported CI execution history. The report therefore uses labeled screenshot placeholders and describes workflow configuration without fabricated screenshots or pass rates. This makes the remaining documentation work explicit and preserves academic integrity.

7.11 Improvement Roadmap

Short-term priorities are repairing the Maven wrapper, expanding Angular tests, adding API smoke tests, and capturing controlled UI evidence. Medium-term priorities are transactional event publication, schema governance, immutable deployments, backups, alerting, and security scanning. Longer-term work could include validated demand forecasting, hospital-system integration, mobile clients, accessibility testing, and real-world usability evaluation under appropriate governance.

Conclusion

Qatra includes a credible industrialization foundation: CI and deployment workflows, containers, k3s manifests, and a complete observability stack. At the same time, local build limitations, frontend test scarcity, and missing execution evidence prevent exaggerated claims. This distinction between implemented foundation and verified result is essential for responsible engineering reporting.

GENERAL CONCLUSION

This end-of-studies project addressed the fragmentation of blood-donation and emergency coordination through the design and implementation of Qatra. The platform unifies donor onboarding, health and availability information, center discovery, appointment scheduling, staff operations, donation outcomes, emergency requests, candidate matching, notification, governance, and audit-oriented analytics.

The solution combines an Angular 20 frontend with Java 21 and Spring Boot backend services. A domain-oriented donation service contains the principal business contexts, while a separate notification service supports asynchronous and real-time delivery concerns. PostgreSQL provides relational persistence, Kafka supports event exchange, and Redis participates in the runtime environment. Docker, k3s, Nginx, GitHub Actions, Prometheus, Grafana, Loki, Promtail, and Jaeger extend the project from application code to deployable and observable infrastructure.

Kanban provided a suitable delivery method for work that continuously crossed analysis, backend, frontend, infrastructure, and documentation. Grouping the completed flow into six functional increments made dependencies visible without misrepresenting the method as time-boxed Scrum.

The project's principal engineering contribution is the coherent treatment of emergency coordination. Matching is not reduced to broadcasting a message: compatibility, eligibility, availability, distance, deadline, preference, and prior contact all affect candidate selection. Notification is treated as a distributed delivery problem involving events, persistence, retries, deduplication, WebSocket authorization, privacy, and auditability.

The available repositories also reveal limitations. Backend test structures exist, but local execution was blocked by the Maven wrapper startup behavior. Frontend automated coverage is minimal. The archive does not contain application screenshots or CI execution exports. Rather than hiding these gaps, the report records them and proposes concrete corrections.

Future work should prioritize reproducible builds, frontend and end-to-end tests, transactional event publication, secure releases, backup validation, security testing, and accessibility. Later partnerships could support usability studies, governed hospital integration, and validated forecasting while preserving privacy, clinical responsibility, and trust.

BIBLIOGRAPHY

- [1] Keyrus, *Corporate Website and Group Presentation*, <https://keyrus.com/>, accessed June 2026.
- [2] S. Newman, *Building Microservices*, 2nd ed., O'Reilly Media, 2021.
- [3] A. Cockburn, "Hexagonal Architecture," Alistair Cockburn, 2005.
- [4] Apache Software Foundation, *Apache Kafka Documentation*, <https://kafka.apache.org/documentation/>, accessed June 2026.
- [5] Broadcom, *Spring Boot Reference Documentation*, <https://docs.spring.io/spring-boot/>, accessed June 2026.
- [6] Google, *Angular Documentation*, <https://angular.dev/>, accessed June 2026.
- [7] PostgreSQL Global Development Group, *PostgreSQL Documentation*, <https://www.postgresql.org/docs/>, accessed June 2026.
- [8] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, 2012.
- [9] Docker Inc., *Docker Documentation*, <https://docs.docker.com/>, accessed June 2026.
- [10] The Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>, accessed June 2026.
- [11] GitHub, *GitHub Actions Documentation*, <https://docs.github.com/actions>, accessed June 2026.
- [12] Prometheus Authors, *Prometheus Documentation*, <https://prometheus.io/docs/>, accessed June 2026.
- [13] Grafana Labs, *Grafana Documentation*, <https://grafana.com/docs/>, accessed June 2026.
- [14] Grafana Labs, *Loki Documentation*, <https://grafana.com/docs/loki/>, accessed June 2026.
- [15] The Jaeger Authors, *Jaeger Documentation*, <https://www.jaegertracing.io/docs/>, accessed June 2026.
- [16] Kanban Guides, *The Kanban Guide*, <https://kanbanguides.org/>, accessed June 2026.
- [17] C. Richardson, "Transactional Outbox Pattern," *Microservices Patterns*, <https://microservices.io/patterns/data/transactional-outbox.html>, accessed June 2026.
- [18] Mosaique FM, "Don du sang: une plateforme pour les rendez-vous," <https://www.mosaiquefm.net/fr/national-tunisie/1365469/don-du-sang-une-plateforme-pour-les-rendez-vous>, accessed August 2026.

- [19] American Red Cross, *Blood Donor App*, <https://www.redcrossblood.org/blood-donor-app.html>, accessed August 2026.
- [20] Meta, “Over 100 Million People Have Signed Up for Local Blood Donation Notifications,” <https://about.fb.com/news/2021/06/100-million-people-signed-up-for-blood-donation-notifications/>, accessed August 2026.
- [21] World Health Organization, *Blood Donor Selection: Guidelines on Assessing Donor Suitability for Blood Donation*, 2012, ISBN 978-92-4-154851-9, [WHO publication page](#), accessed August 2026.
- [22] World Health Organization, *Screening Donated Blood for Transfusion-Transmissible Infections and Blood-Group Testing*, [WHO donation-testing guidance](#), accessed August 2026.
- [23] American Red Cross, *Blood Types and Red-Cell Compatibility*, [official compatibility guidance](#), accessed August 2026.
- [24] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, 2016, [official consolidated text](#), accessed August 2026.
- [25] OWASP Foundation, *OWASP API Security Top 10 – 2023*, [official project page](#), accessed August 2026.
- [26] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum*, November 2020, [official guide](#), accessed August 2026.



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454